

A Survey on Role-Oriented Network Embedding

Pengfei Jiao¹, Xuan Guo¹, Ting Pan, Wang Zhang¹, Yulong Pei, and Lin Pan¹

Abstract—Recently, Network Embedding (NE) has become one of the most attractive research topics in machine learning and data mining. NE approaches have achieved promising performance in various graph mining tasks including link prediction and node clustering and classification. A wide variety of NE methods focus on the proximity of networks, they learn community-oriented embedding for each node, where the corresponding representations are similar if two nodes are closer to each other in the network. Meanwhile, there is another type of structural similarity, i.e., role-based similarity, which is completely different from and complementary to the proximity. In order to preserve the role-based structural similarity, the problem of role-oriented NE is raised. However, compared to the community-oriented NE, there are only a few role-oriented embedding approaches proposed recently. Although less explored, considering the importance of roles in analyzing networks and many applications that role-oriented NE can shed light on, it is necessary and timely to provide a comprehensive overview of existing role-oriented NE methods. In this review, we first clarify the differences between community-oriented and role-oriented network embedding. Afterward, we propose a general framework for understanding role-oriented NE and a two-level categorization to better classify existing methods. Then, we select some representative methods according to the proposed categorization and briefly introduce them by discussing their motivation, development, and differences. Moreover, we conduct comprehensive experiments to empirically evaluate these methods on a variety of role-related tasks including node classification and clustering (role discovery), top-k similarity search, and visualization using some widely used synthetic and real-world datasets. Finally, we further discuss the research trend of role-oriented NE from the perspective of applications and point out some potential future directions.

Index Terms—Network embedding, role discovery, structural similarity, unsupervised learning, experimental analysis

1 INTRODUCTION

NETWORK or graph is usually used to model the complex interaction relations in real-world data and systems [1], [2], e.g., transportation, social pattern, cooperative behavior and metabolic phenomenon. By convention, the network (or graph) is usually abstracted as some nodes and their complicated and elusive links. To understand such data, network analysis can help to explore the organization, analyze the structure, predict the missing links and control the dynamics in complex systems. For a long time, researchers have proposed specially designed methods and models for different graph mining tasks, such as preference mechanism, hierarchical structure and latent space model for link prediction [3]; grouping or aggregation, bit compression and influence based for network summarization [4]; generalized threshold model, independent cascade model and linear Influence Model for information diffusion [5]. Among the core issues and applications of network analysis and

graph mining, clustering [6], dividing the nodes into distinct or overlapping groups, has attracted the most interest from different domains including machine learning and complex networks.

The field of network clustering has two main branches: *community detection* [7], [8], [9] and *role discovery* [10]. Community detection, the currently dominant research branch of clustering, is devoted to finding common groups in which nodes interact more intensively than outside [11]. However, role discovery, which has a long research history in sociology [12] but had been inconspicuous in network science, groups the nodes based on the similarity of their structural patterns [13], such as the bridge or hub nodes [14]. In general, nodes in the same community are likely to be connected, while nodes in the same role may be unconnected and often far away from each other. Since their rules on dividing nodes are fundamentally different, these two branches are usually considered to be orthogonal problems. A variety of algorithms and models are proposed for both branches. For community detection, modularity optimization [15], [16], statistical model [17], [18], non-negative matrix factorization [19], [20], [21], [22] and deep learning methods [23], [24] are developed and show crucial influence for other tasks and applications, such as recommendation [25], [26] and identifying criminal gangs [27]. Some surveys on community detection can be seen in [11], [28], [29]. For role discovery, traditional methods are usually graph-based and related to some equivalence, such as the structural [30], regular [31] and stochastic equivalence [32], [33]. Blockmodels [34] and mixed-membership stochastic blockmodels [35] are the important and influential methods based on the graph. Besides, there are also some combinatorial or heuristic methods [36] for this problem.

- Pengfei Jiao is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China. E-mail: cspjiao@gmail.com.
- Xuan Guo, Ting Pan, and Wang Zhang are with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China. E-mail: {guoxuan, tingpan, wangzhang}@tju.edu.cn.
- Yulong Pei is with the Department of Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven 5600 MB, The Netherlands. E-mail: y.pei@tue.nl.
- Lin Pan is with the School of Marine Science and Technology, Tianjin University, Tianjin 300072, China. E-mail: linpan@tju.edu.cn.

Manuscript received 3 June 2021; revised 21 Nov. 2021; accepted 25 Nov. 2021. Date of publication 30 Nov. 2021; date of current version 9 July 2022.

This work was supported in part by the National Natural Science Foundation of China under Grant 61902278.

(Corresponding author: Lin Pan.)

Recommended for acceptance by P.D. Jin, G. Song, W. Wang, P.S. Yu, J. Han. Digital Object Identifier no. 10.1109/TBDDATA.2021.3131610

In recent years, network embedding (NE) has become the focus of studying graph structure and has been demonstrated to achieve promising performance in many downstream tasks, e.g., node classification and link prediction. The motivation of NE is to transform the network data into vector-format representations in a latent space, and these representations are capable of preserving the topological structure and properties of the original network. On the whole, current methods for NE can be categorized into two types: shallow and deep learning (here we focus on unsupervised NE approaches without explicit mentioning). The former includes matrix factorization and random walk-based methods. The goal of matrix factorization methods is to learn node embedding via low-rank approximation and approaching the adjacency matrix or higher-order similarity of the network, such as the singular value decomposition, non-negative matrix factorization and NetSMF [37]. With the different random walk strategies, a series of methods have been proposed to optimize the co-occurrence probability of nodes and learn effective embeddings. The latter is mainly rooted in autoencoder and graph convolutional networks. These methods generally consist of the encoder, similarity function and the decoder. There are also some attributes, characteristics and constraints that can be combined to enhance the embedding, and VGAE [38], GAT [39] and GraphSAGE [40] are some representatives of such methods. There are also some other types of embedding approaches, e.g., the latent feature model, but discussion on general NE methods is beyond our scope. We refer the interested readers to some recent survey papers on NE [41], [42], [43], [44], [45], [46], [47], [48].

However, most of these methods, whether or not for network clustering, are designed for modeling the *proximity*, i.e., the embedding vectors are community-oriented. They fail to capture the structural similarity, or the role information [49]. Therefore, it raises several inherent challenges in the research of role-oriented network embedding. First and most importantly, two nodes with structural similarity usually have nothing to do with their distance, which makes it difficult to define the loss function effectively. Second, strict role definitions, such as some definitions based on equivalence, are difficult to implement in real-world networks, especially large-scale networks. Third, the distribution of nodes with the same role in the network is very complex and interaction patterns between different roles are unknown.

There are still some scattered methods being proposed one after another recently. These methods use various embedding mechanisms. Struc2vec [50] leverages random walks on graphs in which edges are weighted based on structural distances. DRNE develops a deep learning framework with layer-normalized [51] LSTM model to learn regular equivalence. REACT [52], generating embeddings via matrix factorization, focuses on capturing both community and role properties. Though the number of diverse role-oriented embedding methods is gradually increasing, there is still a lack of systematic understanding of role-oriented network embedding. Besides, we also lack a taxonomy for deep thinking about this problem. Meanwhile, there is short of performance and efficiency comparison of current methods. All these limit the development and application of role embedding.

Therefore, in this survey, we systematically analyze role-oriented network embedding and the analysis can help to understand the internal mechanism of current methods. First, we propose a two-level categorization scheme for existing methods based on the embedding mechanism of current methods and models. Furthermore, we evaluate selected embedding methods from the perspectives of both efficiency and effectiveness on different tasks related to role discovery. Specifically, we conduct comprehensive experiments on some representative methods on running time (efficiency), node classification and clustering (role discovery), top-k similarity search and visualization with widely used benchmark networks. Last, we summarize the applications, challenges and future directions of role-oriented network embedding.

Some surveys on network embedding, community detection, role discovery, and deep learning on graphs have been conducted. Our survey has several essential differences compared to these works [11], [28], [29] that mainly study the problem of community detection with different focuses from the perspective of network analysis and machine learning. [10] is a seminal work in reviewing the development and methodology of role discovery. However, this survey is relatively outdated where more advanced methods, e.g., deep learning-based methods, have not been discussed. Besides, these surveys focus on the methods specific for community or role tasks, while our work studies roles with a focus on network embedding approaches preserving the role information. The surveys [41], [42], [47], [48] are influential works on network embedding from different principles. However, they all focus on community-oriented methodology. Similarly, some graph embedding¹ reviews [45], [46] have nothing to do with role-oriented embedding, except for some technologies. Meanwhile, some surveys such as [44] and [43] introduce the effective deep learning framework and methods on graphs or networks. They focus on general problems of how to use machine learning on networks and are less relevant to our problem. One relevant work is [49], which clarifies the difference between the community-oriented and role-oriented network embedding for the first time. It also proposes the normal mechanisms which can help to understand if a method is designed for community detection or role discovery. However, it does not systematically discuss the series of role-oriented NE methods: some advanced methods have been ignored, some introduced works are not used for role discovery or role-related tasks. Moreover, it does not evaluate methods empirically by analyzing the relevant data, tasks and performance. Another recent work [53], which introduces some structural node embedding methods and evaluates them empirically, is the most relevant to our work. In analysis, they focus on analyzing the relationships between NE methods and equivalence. In evaluation, they evaluate the discovered roles on direct tasks such as role classification and clustering. In contrast, we concentrate on analyzing the advantages and disadvantages of different role-oriented approaches using a new two-level categorization from the analysis perspective. We conduct more comprehensive experiments to evaluate different methods w.r.t. both

1. We do not distinguish the difference between network embedding and graph embedding.

TABLE 1
Main Notations

Notation	Definition
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	the network/graph with node set $\mathcal{V}$ and edge set $\mathcal{E}$
$\mathcal{N}_i^k$	the set of v_i 's k -hop reachable neighbors
$\mathcal{G}_i^k$	the subgraph induced by v_i and $\mathcal{N}_i^k$
d_i	the degree of node v_i
s_{ij}	the shortest path between v_i and v_j
$\mathbf{X}$	attribute matrix
$\mathbf{I}$	identity matrix
$\mathbf{H}$	embedding matrix
$\mathbf{F}_{meth}$	the feature matrix extracted by or in method $meth$
$\mathbf{S}_{meth}$	the similarity matrix obtained by or in method $meth$
$\circ, \langle \langle \cdot \rangle \rangle$	the concatenation operator

*For convenience, method notation m is omitted in some descriptions.

efficiency and effectiveness in role discovery and downstream tasks including running time, classification, clustering, visualization, and top-k similarity search.

To sum up, our survey has several contributions as follows.

- We first show the summary of role-oriented network embedding and discuss the relationship and differences between it and the community-oriented.
- We propose a two-level categorization schema of current role-oriented embedding methods and briefly describe their formalization, mechanism, task, connection and difference.
- We provide full experiments of popular methods of each type on different role-oriented tasks and detailed comparison on effectiveness and efficiency.
- We share all the open-source code and widely used network datasets on Github and point out the development and questions of role-oriented network embedding.

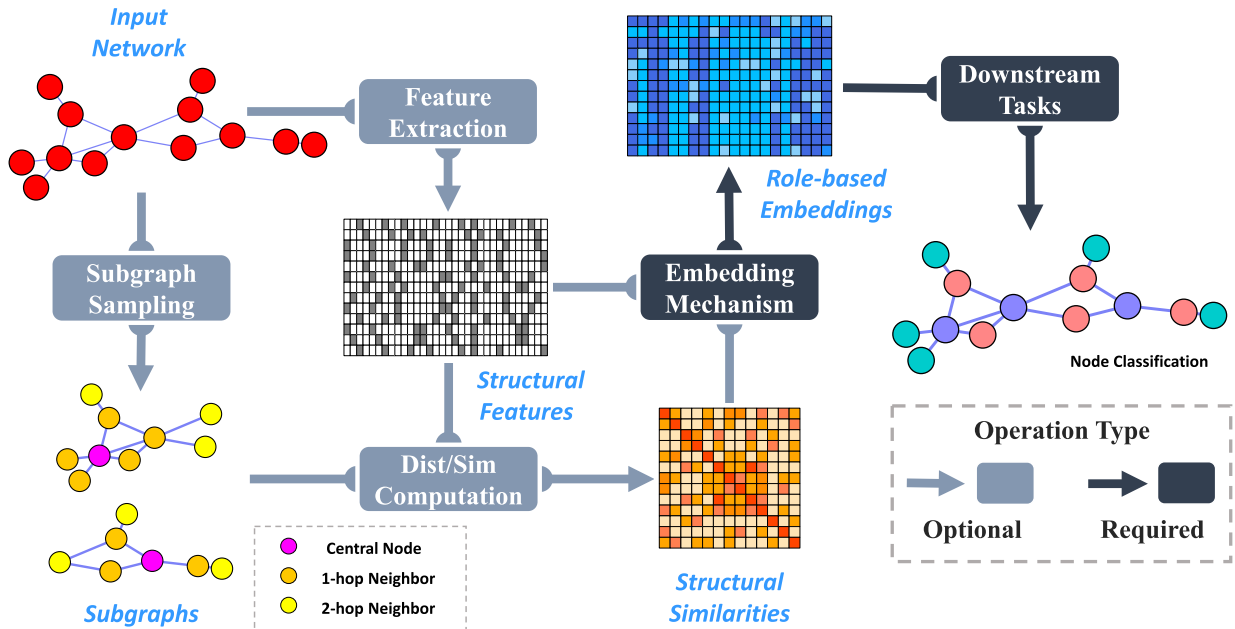


Fig. 1. The common framework of role-oriented network embedding methods includes two main step: structural property extraction and embedding. The former one can be accomplished via a variety of ways of which some are feature-based and some are similarity-based methods. On the extracted properties, role-oriented embedding methods then employ some specific embedding mechanisms to generate embeddings. Note that the discussed role-oriented methods are unsupervised. Thus, though the generated embeddings can be applied on some downstream tasks with ground truth, the whole process of embedding generation has no interaction with the target tasks.

Authorized licensed use limited to: TIANJIN UNIVERSITY. Downloaded on August 21, 2026 at 05:17:53 UTC from IEEE Xplore. Restrictions apply.

2 NOTATIONS AND FRAMEWORK

In this section, we give formal definitions of basic graph concepts and role-oriented network embeddings. In Table 1, we summarize the main notations used throughout this paper. Then, we propose a unified framework for understanding the process of role oriented network embedding.

Definition 1 (Network). A network is denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of m edges. An edge $e_{ij} = (v_i, v_j) \in \mathcal{E}$ denotes the link between node v_i and v_j .

In usual, a network $\mathcal{G}$ is represented by an weight matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. If $e_{ij} \in \mathcal{E}$, $\mathbf{A}_{ij} > 0$ ($\mathbf{A}_{ij} = 1$ for an unweighted network and $\mathbf{A}_{ij} = \mathbf{A}_{ji}$ for an undirected network), otherwise $\mathbf{A}_{ij} = 0$. Some networks may have an attribute matrix $\mathbf{X} \in \mathbb{R}^{n \times x}$ whose i th row represents attributes of v_i . For an undirected network, denote the degree of node v_i as $d_i = \sum_j \mathbf{A}_{ij}$, and we have the degree matrix $\mathbf{D} = \text{Diag}(d_1, \dots, d_n)$. $\mathbf{L} = \mathbf{D} - \mathbf{A}$ is called the Laplacian matrix, it can be decomposed as $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where $\mathbf{\Lambda} = \text{Diag}(\lambda_1, \dots, \lambda_n)$ is the matrix of eigenvalues satisfying $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.

Denote the k -hop ($k > 0$) reachable neighbor set of node v_i as $\mathcal{N}_i^k$ (k is omitted when $k = 1$), where the shortest path between v_i and each node $v_j \in \mathcal{N}_i^k$ is less than or equal to k . For a directed network, use d_i^+ , d_i^- , $\mathcal{N}_i^{k+}$ and $\mathcal{N}_i^{k-}$ to represent the out/in-degree and k -hop reachable out/in-neighborhood of v_i respectively. Unless otherwise stated, a model is discussed on unweighted undirected networks without attributes in later part of this paper.

Definition 2 (Motif/Graphlet). A motif/graphlet $\mathcal{M} = (\mathcal{V}_{\mathcal{M}}, \mathcal{E}_{\mathcal{M}})$ is a small connected subgraph representing particular patterns of edges on several nodes. The pattern can be

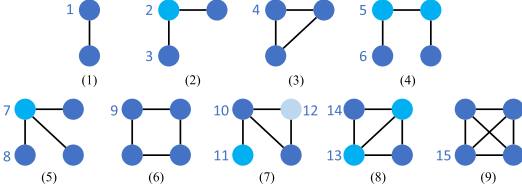


Fig. 2. Motifs and orbits (denoted by numbers) with size 2-4 nodes.

repeated in or across networks, i.e., many subgraphs can be sampled from networks to be isomorphic to it. Nodes being automorphic to each other, i.e., having the same connectivity patterns, are in the same orbits.

For unweighted networks, there are 9 motifs and 15 orbits with size 2-4 nodes as shown in Fig. 2. Because of their ability to model the smallest but most fundamental connectivity patterns, motifs are widely used for capturing structural similarities and discovering roles.

Definition 3 (Network Clustering). A clustering $\mathcal{R} = \{\mathcal{R}_1, \dots, \mathcal{R}_k\}$ of network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a group of node sets satisfying $\forall 1 \leq i \leq k: \mathcal{R}_i \neq \emptyset, \mathcal{R}_i \subset \mathcal{V}$ and $\bigcup_{i=1}^k \mathcal{R}_i = \mathcal{V}$. In this paper, we discuss about hard clustering, i.e., $\forall 1 \leq i < j \leq k: \mathcal{R}_i \cap \mathcal{R}_j = \emptyset$. If $\mathcal{R}_i \cup \mathcal{R}_j \neq \emptyset$, it is usually called overlapping or soft clustering.

For community detection, each set $\mathcal{R}_i$ is a tightly interconnected collection of nodes. And for role discovery, it is usually composed of unconnected nodes having similar structural patterns or functions. So every network clustering algorithm is committed to achieve the clustering results under different goal constraints. However, there is no common understanding of role equivalence or similarity, leading to multifarious definitions of equivalence and designs of similarity computation. For example, two nodes are automorphic equivalent [54] as the subgraphs of their neighborhood are isomorphic, while regular equivalence [55] means that if the nodes have the same roles, their neighbors have the same roles.

Definition 4 (Network Embedding). Network Embedding is a process to map nodes of network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ to low-dimensional embeddings $\mathbf{H} \in \mathbb{R}^{n \times r}$ with $r \ll n$. In general, for nodes v_i and v_j , if they are similar in the network, their embedding vectors $\mathbf{H}_i$ and $\mathbf{H}_j$ will be close in the low dimensional space.

The node embeddings can be taken for different network tasks. If we focus on community detection or link predication, we want to discriminate by embeddings whether the nodes are connected or likely to be connected. However, for role discovery, the embeddings should reflect some structural patterns, including local properties like subgraph isomorphism, global properties like regular equivalence and higher-order properties like motifs.

Based on the discussion and notations above, here we first propose a unified framework for understanding the role-oriented network embedding. To our knowledge, it can cover almost all the existing methods and models in a unified way. The framework is illustrated in Fig. 1. The structure of networks is discrete, but embeddings are usually designed to lie in continuous space. Thus, role-oriented

embedding methods always take the following two steps to capture structural properties and generate embeddings respectively to bridge the gulf between two spaces:

(1) *Structural Property Extraction.* The ways to extract structural information are diverse. Some methods such as RolX [56] and DRNE [51] leverage some primary structural features such as node degrees and triangle numbers. A part of these methods such as SPINE [57] will continue to transform the features into distances or similarities. There are also some methods capturing similarity between node-centric subgraphs. For example, struc2vec [50] computes structural distances between k -hop subgraphs based on degree sequences of the subgraphs. SEGK [58] employs one graph isomorphism test skill called graph kernel on subgraphs. As the result of these hand-craft process, these structural properties are contained in interim features or pair-wise similarities. The extracted structural properties can be overall divided into three types: local, global, and higher-order. As the names imply, local properties (e.g., node degree) are the statistics and measures calculated on neighborhoods and global properties (e.g., PageRank) are those calculated on the whole network. Higher-order properties (e.g., motif) describe how a group of nodes are connected to each other.

(2) *Embedding.* The extracted properties are then mapped into embedding space via different mechanisms. In the process of embedding, the structural properties are used as inputs or training guidance. For example, RolX [56] and SEGK [58] apply matrix factorization on feature matrix and similarity matrix respectively, as they implicitly or explicitly reflect whether nodes are structurally similar. Struc2vec [50] and SPINE [57] leverage word embedding methods to the similarity-biased random walks. DRNE [51] utilizes LSTM [59], a deep learning method, on degree-ordered node embedding sequences to capture regular equivalence with a degree-guided regularizer. Both matrix factorization and random walk-based embedding are shallow embedding mechanisms [47]. Though their detailed procedures of reconstructing structural similarity are different, they generate embeddings by updating lookup tables similarly. Compared with the shallow mechanisms, deep learning methods share the same encoders among nodes with a number of parameters so that they are good at non-linear mapping and computationally efficient.

As the embeddings capture crucial structural properties, they can be used on the downstream tasks such as role-based node classification and visualization. With this framework, we generalize the process of role-oriented embedding. However, as we can learn from Fig. 1, the core of designing role-oriented embedding methods is the way to extract structural properties. In contrast, the embedding mechanisms for mapping structural features/similarities into low-dimensional continuous vector space are much more regular. Thus, we introduce the popular methods in the next section from the perspective of embedding mechanisms.

3 ALGORITHM TAXONOMY

In this section, we introduce these approaches categorized according to their embedding mechanisms. In detail, we propose a two-level categorization for these popular methods. We divide these into three categories, including matrix

TABLE 2
A Summary of Role-Oriented Embedding Methods

Method	Embedding Mechanism		Extracted Structure	Conducted Tasks				Time Complexity
				Vis	CLF/CLT	ER/NA/SS	LP	
RolX	matrix factorization (Sec.3.1)	on structural feature matrix (Sec.3.1.1)	local	✓	✓	✗	✗	$O(\varpi m + \varpi' n)$
GLRD			local	✗	✗	✓	✗	$O(\varpi m + \varpi' n)$
RIDeRs			global	✓	✓	✓	✗	$O(\varpi n)$
GraphWave			global	✓	✓	✗	✗	$O(\varpi m)$
HONE			higher-order	✓	✗	✓	✓	$O(\varpi m + \varpi' n)$
xNetMF		on structural similarity matrix (Sec.3.1.2)	local	✗	✗	✓	✗	$O(n(\varpi d_{avg}^2 + \varpi' d_{max}))$
EMBER			local	✗	✓	✓	✗	$O(n(\varpi d_{avg}^2 + \varpi' d_{max}))$
SEGK			higher-order	✓	✓	✓	✗	$O(\varpi n^2 + \varpi' n)$
REACT			global	✗	✓	✗	✗	$O(\varpi n^2 + \varpi' n)$
SPaE			higher-order	✓	✓	✗	✗	$O(\varpi n^2)$
struc2vec	random walk-based methods (Sec.3.2)	on similarity-biased random walks (Sec.3.2.1)	higher-order	✓	✓	✗	✗	$O(\varpi n^3 + \varpi' n)$
SPINE			global	✗	✓	✗	✗	$O(\varpi n^2 + \varpi' n)$
struc2gauss			global	✓	✓	✗	✗	$O(\varpi n^2 + \varpi' n)$
Role2Vec		on feature-based random walks (Sec.3.2.2)	user-defined	✗	✗	✗	✓	$O(\varpi n)$
RiWalk			higher-order	✗	✓	✗	✗	$O(n(m + \varpi))$
NODE2BITS			local	✗	✗	✓	✗	$O(\varpi m + \varpi' n)$
DRNE	deep learning (Sec.3.3)	via structural information reconstruction / guidance (Sec.3.3.1)	local	✓	✓	✗	✗	$O(\varpi n)$
GAS			local	✓	✓	✗	✗	$O(\varpi m)$
RESO			local	✓	✓	✓	✗	$O(\varpi m + \varpi' n)$
GraLSP			higher-order	✓	✓	✗	✓	$O(\varpi m + \varpi' n)$
GCC			higher-order	✗	✓	✓	✗	$O(\varpi m + \varpi' n)$

The abbreviations of tasks CLF, CLT, LP, ER, NA, SS and Vis denote node classification/clustering, link prediction, entity resolution/network alignment/Top-k similarity search and visualization, respectively.

factorization, random walk-based and deep learning methods at the first level. Furthermore, based on the embedding mechanisms and constraint information, we give a more refined taxonomy as shown in Table 2. At the same time, we also list the tasks which could be served by different methods in their original paper. Next, we will detail how these methods work for capturing structure into embeddings and give a complexity analysis.

3.1 Matrix Factorization

Matrix factorization is the most commonly used method for role-oriented embedding methods. They generate embeddings by factorizing matrices preserving the role similarities between nodes implicitly (i.e., feature matrices) or explicitly (i.e., similarity matrices).

3.1.1 Structural Feature Matrix Factorization

RolX [56]. *RolX* takes the advantages of the feature extraction method ReFeX [60] by decomposing the ReFeX feature matrix $\mathbf{F}_{ReFeX} \in \mathbb{R}^{n \times f}$. ReFeX first computes some primary features such as degree and clustering coefficient for each node. Then it recursively aggregates neighbors' features with sum- and mean-aggregator. In k recursive steps, it can capture very thorough features to express the structure of $(k+1)$ -hop reachable neighborhood. Non-negative Matrix Factorization (NMF) is used for generating embeddings as it is efficient compared with other matrix decomposition methods. The non-negative constraints are adapted to the interpretation of roles. Thus, *RolX* aims to obtain two matrices as follows

$$\min_{\mathbf{H}, \mathbf{M}} \left\| \mathbf{F}_{ReFeX} - \mathbf{H}\mathbf{M} \right\|_F^2, \text{ s.t. } \mathbf{H}, \mathbf{M} \geq 0, \quad (1)$$

where $\mathbf{H} \in \mathbb{R}^{n \times r}$ is the embedding matrix (or role assignment matrix) and the matrix $\mathbf{M} \in \mathbb{R}^{r \times f}$ (role definition matrix) describes the contributions of each role to structural features. r is the number of hidden roles determined by Minimum Description Length (MDL) [61].

GLRD [14]. *GLRD* extends *RolX* by adding different optional constraints to objective function (1). Sparsity constraint $(\forall i, \|\mathbf{H}_i\|_1 \leq \epsilon_H \wedge \|\mathbf{M}_i\|_1 \leq \epsilon_M)$ is defined for more definitive role assignments and definitions while diversity constraint $(\forall i \neq j, \mathbf{H}_i^\top \mathbf{H}_j \leq \epsilon_H \wedge \mathbf{M}_i^\top \mathbf{M}_j \leq \epsilon_M)$ is for reducing the overlapping. $\mathbf{H}^*$ and $\mathbf{M}^*$ are previously discovered role assignments and definitions with which alternativeness constraint $(\forall i \neq j, \mathbf{H}_i^\top \mathbf{H}_j \leq \epsilon_H \wedge \mathbf{M}_i^\top \mathbf{M}_j \leq \epsilon_M)$ can be used for mining roles unknown.

RIDeRs [62]. *RIDeRs* uses ε -equitable refinement (ε ER) to partition nodes into different cells and compute graph-based features. An ε -equitable refinement partition $\pi = \{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K\}$ of $\mathcal{V}$ satisfies the following rule

$$|\deg(u, \mathcal{C}_j) - \deg(v, \mathcal{C}_j)| \leq \varepsilon, \forall u, v \in \mathcal{C}_j, \forall 1 \leq i, j \leq K, \quad (2)$$

where $\deg(u, \mathcal{C}_j) = |\{u | (u, v) \in E \wedge v \in \mathcal{C}_j\}|$ denotes the number of nodes in cell $\mathcal{C}_j$ connected to node v_i . As nodes in the same cell have a similar number of connections to the nodes in another cell, ε ERs could capture some connectivity patterns.

Based on the cells partitioned by ε ERs with a relaxation parameter ε , the feature matrix is defined as $(\mathbf{F}_{\varepsilon ER}^i)_{ij} =$

$|\mathcal{N}_i \cap \mathcal{C}_j|$. After pruning and binning process, the feature matrices for all $1 \leq \varepsilon \leq \lfloor d_{avg} \rfloor$ are concatenated as the final feature matrix $\mathbf{F}_{\varepsilon ER}$. Finally, like RoIX and GLRD, NMF is applied for embedding generation while right sparsity constraint (on $\mathbf{M}$) is optional for more definitive role representations.

GraphWave [63]. *GraphWave* treats graph diffusion kernels as probability distributions over networks and gets embeddings by using characteristic functions of the distributions. Specifically, take the heat kernel $g_\varsigma(\lambda) = e^{-\lambda\varsigma}$ with scaling parameter ς as an example, the spectral graph wavelets $\Psi \in \mathbb{R}^{n \times n}$ are defined as

$$\Psi = \mathcal{I} \mathbf{U} \text{Diag}(g_\varsigma(\lambda_1), \dots, g_\varsigma(\lambda_n)) \mathbf{U}^\top, \quad (3)$$

where $\mathcal{I}$ is the one-hot encoding matrix on $\mathcal{V}$ and the scaling parameter ς is omitted. The i th row Ψ_i represents the resulting signal from a Dirac signal around node v_i . Considering the empirical characteristic function

$$\varphi_i(t) = \frac{1}{n} \sum_{j=1}^n e^{it\Psi_{ij}}, \quad (4)$$

where i denotes the imaginary number, v_i 's embedding vector $\mathbf{H}_i$ is generated by concatenating pairs of $\text{Re}(\varphi_i(t))$ and $\text{Im}(\varphi_i(t))$ at r evenly spaced points $t_1, \dots, t_r$.

For high efficiency, Chebyshev polynomial approximation [64] is used when computing Eq. (3) so that the total computational cost is linear in the number of edges.

HONE [65]. *HONE* constructs weighted motif graphs in which the weight of an edge is the count of the co-occurrences of the two endpoints in a specific motif. For a motif represented by its weighted motif adjacency matrix $\mathbf{A}_{\mathcal{M}_i}$, *HONE* characterizes the higher-order structure by deriving matrices from its k -step matrices $\mathbf{A}_{\mathcal{M}_i}^k$. These new matrices are designed by imitating some popular matrices based on normal adjacency matrix such as transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ and Laplacian matrix $\mathbf{L} = \mathbf{D} - \mathbf{A}$. Here we use $(\mathbf{F}_{HONE})_{\mathcal{M}_i}^{(k)}$ to denote the derived matrices. Then the k -step embeddings can be learned as

$$\arg \min_{\mathbf{H}_{\mathcal{M}_i}^{(k)}, \mathbf{M}_{\mathcal{M}_i}^{(k)}} \mathbb{D}_{Breg}(\mathbf{F}_i^{(k)} | \Psi(\mathbf{H}_{\mathcal{M}_i}^{(k)} \mathbf{M}_{\mathcal{M}_i}^{(k)})), \quad (5)$$

where $\mathbb{D}_{Breg}$ is the Bregman divergence and $\Psi(\cdot)$ is a matching function.

The global embeddings are generated by minimizing the following objective

$$\min_{\mathbf{H}, \mathbf{M}} \left\| \mathbf{F}_{HONE} - \mathbf{H} \mathbf{M} \right\|_F^2, \quad (6)$$

where $\mathbf{F}_{HONE}$ is obtained by concatenating the $\mathbf{H}_{\mathcal{M}_i}^{(k)}$ with all the considered motifs and steps. If necessary, attributes diffused by a transition matrix based on different motifs and steps can be added into $\mathbf{F}_{HONE}$.

Remark. Aforementioned methods assume that nodes in similar roles have similar structural features. Thus, they apply matrix factorization on the feature matrices to obtain role-based representations. RoIX, GLRD and RIDeRs directly get embeddings which give soft role assignment by factorizing feature matrices. As the feature matrices are usually lower

dimension, these methods are quite efficient. *GraphWave* uses eigen-decomposition and empirical characteristic function to characterize the structural patterns of each node, leading to robust embeddings but high computation cost. The weighted motif adjacency matrices in *HONE* capture higher-order proximities actually, while they can obtain structural information because each matrix represents one motif.

3.1.2 Structural Similarity Matrix Factorization

xNetMF [66]. *xNetMF* is an embedding method designed for an embedding-based network alignment approach REGAL. It first obtains a node-to-node similarity matrix $\mathbf{S}_{REGAL}$ based on both structures and attributes

$$\mathbf{S}_{ij} = \exp(-\gamma_s \text{dist}_s(v_i, v_j) - \gamma_a \text{dist}_a(v_i, v_j)), \quad (7)$$

where $\text{dist}_s(v_i, v_j)$ and $\text{dist}_a(v_i, v_j)$ are structure-based and attribute-based distance between node v_i and node v_j while γ_s and γ_a are balance parameters of the two distances. $\text{dist}_s(v_i, v_j)$ is the euclidean distance between node features. And $\text{dist}_a(v_i, v_j)$ counts different attributes between nodes, i.e., $\text{dist}_a(v_i, v_j) = |\{a | \mathbf{X}_{ia} \neq \mathbf{X}_{ja} \wedge 1 \leq a \leq x\}|$. The feature matrix $\mathbf{F}_{REGAL}$ is defined by counting nodes with the same logarithmically binned degree in each node's k -hop reachable neighborhood as follows

$$\begin{aligned} \mathbf{F}_{ic}^k &= |\mathcal{D}_{i,c}^k| = |\{v_j \in \mathcal{N}_i^k | \lfloor \log_2 d_j \rfloor = c\}|, \\ \mathbf{F}_i &= \sum_{k=1}^K \delta^k \mathbf{F}_i^k \end{aligned} \quad (8)$$

where $\delta^k \in (0, 1]$ is a discount factor for lessening the importance of higher-hop neighbors.

Then on computed $\mathbf{S}$, matrix factorization methods can be applied for obtaining embedding matrix $\mathbf{H}$ satisfying $\mathbf{S} \approx \mathbf{H} \mathbf{M}^\top$. As the high dimension and rank of $\mathbf{S}$ lead to high computation, an implicit matrix factorization approach extending Nyström method [67] is proposed as follows:

- 1) Select $r \ll n$ nodes as landmarks randomly or based on node centralities.
- 2) Compute a node-to-landmark similarity matrix $\mathbf{C} \in \mathbb{R}^{n \times r}$ with Eq.(7) and extract a landmark-to-landmark similarity matrix $\mathbf{B} \in \mathbb{R}^{r \times r}$ from $\mathbf{C}$.
- 3) Apply Singular Value Decomposition on the pseudoinverse of $\mathbf{B}$ so that $\mathbf{B}^\dagger = \mathbf{V} \Sigma \mathbf{Y}^\top$.
- 4) Obtain embedding matrix $\mathbf{H}$ by computing and normalize $\mathbf{C} \mathbf{V} \Sigma^{-\frac{1}{2}}$.

With above method, embeddings are actually generated by factorizing a approximation of $\mathbf{S}$, i.e., $\hat{\mathbf{S}} = \mathbf{C}(\mathbf{V} \Sigma \mathbf{Y}^\top) \mathbf{C}^\top$. Meanwhile, the computation cost of matrix factorization can be reduced to $O(nr^2)$, as only a small matrix $\mathbf{B}^\dagger$ is decomposed.

EMBER [68]. *EMBER* is designed for mining professional roles in weighted directed email networks. It defines node outgoing feature matrix $\mathbf{F}_{EMBER}^+$ as

$$\begin{aligned} \mathbf{F}_{ic}^{k+} &= \sum_{v_j \in \mathcal{D}_{i,c}^{k+}} \text{pw}(\mathcal{P}_{v_i \rightarrow v_j}^{k+}), \\ \mathbf{F}_i^+ &= \sum_{k=1}^K \delta^k \mathbf{F}_i^{k+} \end{aligned} \quad (9)$$

where $\text{pw}(\mathcal{P}_{v_i \rightarrow v_j}^{k+})$ denotes the product of all edge weights in a k -step shortest outgoing path $\mathcal{P}_{v_i \rightarrow v_j}^{k+}$. The incoming feature matrix $\mathbf{F}$ is defined similarly. By concatenating the incoming and outgoing feature matrices, the final feature matrix $\mathbf{F}_{\text{EMBER}} = [\mathbf{F}^+, \mathbf{F}^-]$ is obtained. The node-to-node similarities are computed through Eq.(7) without attribute-based distance, i.e., $\mathbf{S}_{ij} = \exp(-\|\mathbf{F}_i - \mathbf{F}_j\|^2)$. EMBER uses the same implicit matrix factorization approach to generate embeddings. Note that if the feature extraction part of EMBER is applied on an undirected unweight network, EMBER will be equivalent to xNetMF without attributes.

SEKG [58]. SEKG leverages graph kernels to compute node structural similarities. To compare the structure more carefully, it computes node similarities with different scales of neighborhood as follows:

$$\mathbf{S}_{ij} = \sum_{k=1}^K \hat{\mathcal{K}}(\mathcal{G}_i^k, \mathcal{G}_j^k) \hat{\mathcal{K}}(\mathcal{G}_i^{k-1}, \mathcal{G}_j^{k-1}), \quad (10)$$

where $\hat{\mathcal{K}}(\mathcal{G}_i^0, \mathcal{G}_j^0) = 1$ and $\hat{\mathcal{K}}$ denotes the normalized kernel which is defined as

$$\hat{\mathcal{K}}(\mathcal{G}, \mathcal{G}') = \frac{\mathcal{K}(\mathcal{G}, \mathcal{G}')}{\sqrt{\mathcal{K}(\mathcal{G}, \mathcal{G})\mathcal{K}(\mathcal{G}', \mathcal{G}')}}, \quad (11)$$

SEKG chooses the shortest path kernel, Weisfeiler-Lehman subtree kernel, or graphlet kernel for practical use of $\mathcal{K}(\cdot, \cdot)$. Then Nystrom method [69] is employed on the factorization of $\mathbf{S}$ for efficient computation and low dimensions of embeddings as follows:

$$\mathbf{H} = \mathbf{S}\mathbf{U}_{[r]}\mathbf{\Lambda}_{[r]}^{-\frac{1}{2}}, \quad (12)$$

where $\mathbf{U}_{[r]}$ denotes the matrix of first r eigenvectors and $\mathbf{\Lambda}_{[r]}$ is the diagonal matrix of corresponding eigenvalues.

REACT [52]. REACT aims to detect communities and discover roles by applying non-negative matrix tri-factorization on RoleSim [70] matrix $\mathbf{S}$ and adjacency matrix $\mathbf{A}$, respectively. RoleSim matrix is developed with the idea of regular equivalence and is a pair-wise similarity matrix computed by iteratively updating the following scores

$$\mathbf{S}_{ij} = (1 - \beta) \max_{\mathbf{M}(v_i, v_j)} \frac{\sum_{(v_i', v_j') \in \mathbf{M}(v_i, v_j)} \mathbf{S}_{i'j'}}{d_i + d_j - |\mathbf{M}(v_i, v_j)|} + \beta, \quad (13)$$

where $\mathbf{M}(v_i, v_j)$ is a matching between the neighborhoods of v_i and v_j , and β ($0 < \beta < 1$) is a decay factor. In addition, $L_{2,1}$ norm is leveraged as the regularization to make the distribution of roles within communities as diverse as possible. Thus, the objective function of REACT is

$$\begin{aligned} \min_{\mathbf{H}_R, \mathbf{M}_R, \mathbf{H}_C, \mathbf{M}_C} & \|\mathbf{S} - \mathbf{H}_R \mathbf{M}_R \mathbf{H}_C^T\|_F^2 \\ & + \|\mathbf{A} - \mathbf{H}_C \mathbf{M}_C \mathbf{H}_C^T\|_F^2 + \gamma_{2,1} \|\mathbf{H}_C^T \mathbf{H}_R\|_{2,1}, \\ \text{s.t. } & \mathbf{H}_R, \mathbf{M}_R, \mathbf{H}_C, \mathbf{M}_C \geq 0, \mathbf{H}_R^T \mathbf{H}_R = \mathbf{I}, \mathbf{H}_C^T \mathbf{H}_C = \mathbf{I}. \end{aligned} \quad (14)$$

where $\mathbf{H}_R/\mathbf{H}_C$ denotes the embedding matrix for roles/communities, and $\mathbf{M}_R/\mathbf{M}_C$ denotes the interaction between roles/communities. $\gamma_{2,1}$ is the weight of regularization.

Orthogonal constraint on embedding matrices is added for increased interpretability.

SPaE [52]. SPaE also tries to capture communities and roles simultaneously. For node structural similarity, it computes cosine similarity between the standardized Graphlet Degree Vectors of nodes, and generates role-based embeddings via the Laplacian eigenmaps method as follows:

$$\max_{\mathbf{H}_R} \mathcal{J}_R = \text{Tr}(\mathbf{H}_R^T \mathbf{L}_S \mathbf{H}_R), \text{ s.t. } \mathbf{H}_R^T \mathbf{H}_R = \mathbf{I}. \quad (15)$$

where $\mathbf{L}_S$ is the symmetric normalized matrix of structural similarity matrix $\mathbf{S}_{\text{SPaE}}$. SPaE obtains community-based embeddings similarly as follows:

$$\max_{\mathbf{H}_C} \mathcal{J}_C = \text{Tr}(\mathbf{H}_C^T \mathbf{L}_A \mathbf{H}_C), \text{ s.t. } \mathbf{H}_C^T \mathbf{H}_C = \mathbf{I}. \quad (16)$$

where $\mathbf{L}_A = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$ is the symmetric normalized adjacency matrix. To map $\mathbf{H}_R$ and $\mathbf{H}_C$ into a unified embedding space, SPaE generates hybrid embeddings by maximizing the following objective function

$$\begin{aligned} \max_{\mathbf{H}_R, \mathbf{H}_C, \mathbf{H}_H} & \mathcal{J}_R + p_R + \gamma(\mathcal{J}_C + p_C), \\ \text{s.t. } & \mathbf{H}_R^T \mathbf{H}_R = \mathbf{I}, \mathbf{H}_C^T \mathbf{H}_C = \mathbf{I}, \mathbf{H}_H^T \mathbf{H}_H = \mathbf{I}. \end{aligned} \quad (17)$$

where $\mathbf{H}_H$ denotes the hybrid embedding matrix and γ is the balance parameter. $p_R = \text{Tr}(\mathbf{H}_R^T \mathbf{H}_H \mathbf{H}_H^T \mathbf{H}_R)$ and $p_C = \text{Tr}(\mathbf{H}_C^T \mathbf{H}_H \mathbf{H}_H^T \mathbf{H}_C)$.

Remark. These methods all explicitly compute structural similarities based on features, e.g., graph kernels, role equivalence, and so on. Most of them have considered the similarities between multiple hops of neighborhoods. Their effectiveness on role discovery depends on the quality of the similarity matrices. One major problem of this kind of methods is the issue of efficiency: computing pair-wise similarity and factorizing the high-dimensional similarity matrix $\mathbf{S} \in \mathbb{R}^{n \times n}$ are time-consuming. So xNetMF, EMBER and SEKG apply the Nystrom method to improve the efficiency as their similarity matrices are Gram matrices [67].

3.2 Shallow Models Using Random Walks

Random walk is a common way to capture node proximity used by network embedding methods [71], [72]. Recently, two strategies have been proposed to adapt random walks to role-oriented tasks: (1) structural similarity-biased random walks makes structurally similar nodes more likely to appear in the same sequence (as shown in Fig. 3b). (2) structural feature-based random walks, e.g., attributed random walks [73], map nodes with similar structural features to the same role indicator and replace ids in random walk sequences with the indicators (see Fig. 3c). The first way can preserve structural similarity into co-occurrence relations of nodes in the walks. While the second way preserves structural similarity into role indicators and may capture the proximity between roles through the co-occurrence relations of the indicators as well.

Usually, language models such as Skip-Gram [74] are applied on generated random walks to map the similarities into embedding vectors [50], [71], [72], [73]. However, some different mapping mechanisms are also employed such as the SimHash [75] used in NODE2BITS [76].

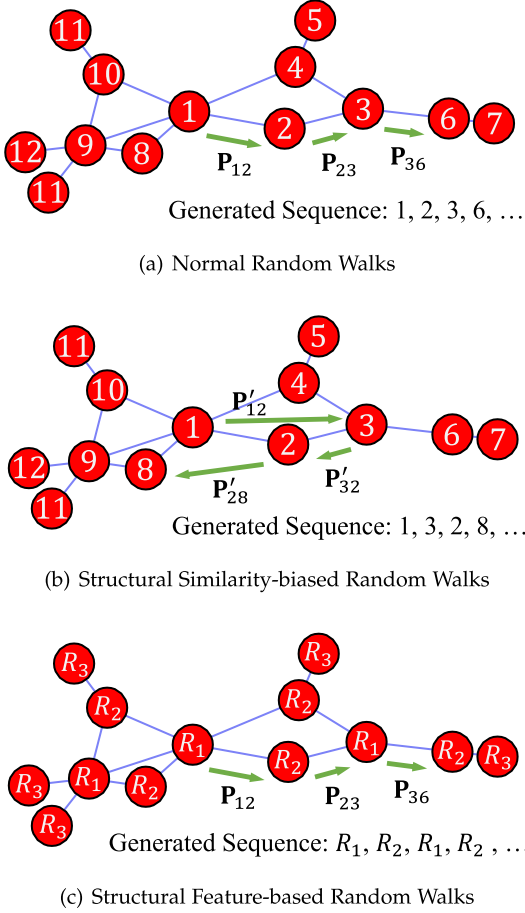


Fig. 3. Different types of random walks. Note that $\mathbf{P}'$ is the biased transition matrix computed based on node structural similarities. R_i is the role indicator mapped from node structural features.

3.2.1 Structural Similarity-Biased Random Walks

struc2vec [50]. *Struc2vec* generates structurally biased id-based contexts via random walks on a hierarchy of constructed complete graphs. Specifically, it first computes structural distances between a pair of nodes as follows:

$$\text{dist}_d^k(v_i, v_j) = \text{dist}_d^{k-1}(v_i, v_j) + \text{DTW}(\mathcal{H}_i^k, \mathcal{H}_j^k), \quad (18)$$

$$0 \leq k \leq k^*$$

where $\text{DTW}(\mathcal{H}_i^k, \mathcal{H}_j^k)$ denotes Dynamic Time Warping (DTW) on the ordered degree sequences of nodes at the exact distance k from v_i and v_j . k^* is the diameter of the $\mathcal{G}$. $\mathcal{H}_i^0 = \{d_i\}$ and $\text{dist}_d^{-1}(v_i, v_j) = 0$.

Then a multi-layer weighted context graph $\mathcal{G}_C$ is built. Each layer is an undirected complete graph $\mathcal{G}_C^k = (\mathcal{V}_C^k, \mathcal{E}_C^k)$. Here we use v_i^k to denote the corresponding node of $v_i \in \mathcal{V}$ in k th-layer. The weight of edge $(v_i^k, v_j^k) \in \mathcal{E}_C^k$ is defined as

$$w_C(v_i^k, v_j^k) = \exp(-\text{dist}_d^k(v_i^k, v_j^k)), k = 0, \dots, k^*. \quad (19)$$

The neighboring layers are connected through directed edges between the corresponding nodes. The edge weights between layers are defined as follows:

$$w_C(v_i^k, v_i^{k+1}) = \log(\Gamma(v_i^k) + e), k = 0, \dots, k^* - 1 \quad (20)$$

$$w_C(v_i^k, v_i^{k-1}) = 1, k = 1, \dots, k^*,$$

where $\Gamma(v_i^k)$ counts the edge $(v_i^k, v_j^k) \in \mathcal{E}_C^k$ whose weight is larger than the average edge weight of $\mathcal{G}_C^k$.

Then id-based random walks can be employed on $\mathcal{G}_C$ and started in layer 0 for context generation of each node. In detail, the walk stays in the current layer with a given probability q_b . And with probability $1 - q_b$, the walk steps across layers. Note that v_i^k with different k have the same id in the context. On the structural context, *struc2vec* leverages Skip-Gram with Hierarchical Softmax to learn embeddings.

The efficiency of *struc2vec* is hampered by the pairwise distance calculations, and the upside is that these calculations can be easily processed in parallel.

SPINE [57]. *SPINE* uses the largest f values of i th row of Rooted PageRank Matrix $\mathbf{\Omega} = (\mathbf{I} - \beta_{RPR})^{-1}$ as v_i 's feature $(\mathbf{F}_{RPR})_i$. β_{RPR} is the probability of stepping to a neighbor, while with probability $1 - \beta_{RPR}$, a walk steps back to the start node. For the inductive setting, *SPINE* computes $\mathbf{F}_i$ via a Monte Carlo approximation, which also reduces the time cost from $O(n^3)$ to being linear to the sample size.

To simultaneously capture structural similarity and proximity, *SPINE* designs a biased random walk method. With probability q_{br} , the walk steps to a structural similar node based on the following transition matrix

$$(\mathbf{P}_{SPINE}^k)_{ij} = \frac{\text{sim}(v_i, v_j)}{\sum_{v_k \in \mathcal{V}, v_k \neq v_i} \text{sim}(v_i, v_k)}. \quad (21)$$

Here $\text{sim}(\cdot, \cdot)$ can be computed via DTW or other methods based on node features. With probability $1 - q_{br}$, normal random walks are applied. Thus, with larger q_{br} , *SPINE* can be more role-oriented. The embeddings are learned through Skip-Gram with Negative Sampling (SGNS). To leverage attributes, the embeddings are generated as

$$\mathbf{H}_i = \sigma \left(\sum_{j=1}^f \mathbf{F}_{ij} \mathbf{X}_j^{<i,f>} \mathbf{W} \right), \quad (22)$$

where $\mathbf{X}^{<i,f>}$ represents the attribute matrix of which the rows correspond to f largest values of $\mathbf{\Omega}_i$. $\mathbf{W}$ is the weight matrix of the multi-layer perceptron (MLP).

struc2gauss [77]. For each node v_i , *struc2gauss* generates a Gaussian distribution: $\mathcal{Z}_i = \text{Gauss}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$ to model both structural similarity and uncertainty. After calculating structural similarity via existing methods such as Role-Sim [70], it samples the top- K most similar nodes for a node as its positive set $\mathcal{S}_i^+$. The positive sampling of *struc2gauss* could be regarded as special random walks with mandatory restart on a star-shaped graph where the star center is the target node and star edges are the most similar nodes. The negative sample set $\mathcal{S}_i^-$ has the same size as $\mathcal{S}_i^+$ and is generated as in the normal random-walk based methods. To push the Gaussian embeddings of similar nodes closer and those of dissimilar nodes farther, *struc2gauss* uses the following max-margin ranking objective

$$\mathcal{L} = \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{S}_i^+} \sum_{v_k \in \mathcal{S}_i^-} \max(0, \xi - \text{sim}(\mathcal{Z}_i, \mathcal{Z}_j) + \text{sim}(\mathcal{Z}_i, \mathcal{Z}_k)), \quad (23)$$

where ξ is the margin parameter to push dissimilar distributions apart, and $\text{sim}(\mathcal{Z}_i, \mathcal{Z}_j)$ is the similarity measure between

distributions of v_i and v_j . There are different similarity measures that can be used, such as logarithmic inner product and KL divergence. For normal tasks, the mean vectors of those Gaussian distributions can be treated as embeddings, i.e., $\mathbf{H}_i = \mu_i$.

Remark. These methods reconstruct the edges between nodes based on structural similarities so that the context nodes obtained by random walks are structurally similar to the central nodes. Compared with SPINE and struc2gauss, struc2vec clearly constructs edges that better represent role information in the multi-layer complete graphs, which leads to better embeddings but higher time and space complexities.

3.2.2 Structural Feature-Based Random Walks

Role2Vec [73]. Role2Vec first maps nodes into several disjoint roles. Logarithmically binning, K-means with factorization and other methods on features and attributes can be chosen for the role mapping $\phi: \mathcal{V} \rightarrow \mathcal{R}$. Motif-based features, such as Graphlet Degree Vectors, are recommended since motif can better capture the high-order structural information.

Then random walks are performed but the ids in generated sequences are replaced with role indicators. With feature-based role context, the language model CBOW can be used for obtaining embeddings of roles. Nodes partitioned into the same role have the same embeddings.

RiWalk [78]. RiWalk designs structural node indicators approximating graph kernels. In a given subgraph $\mathcal{G}_i^k$, the indicator approximating shortest path kernel for node $v_j \in \mathcal{N}_i^k$ is defined as the concatenation of the degrees of v_i and v_j and the shortest path length between them

$$\phi_{SP}^i(v_j) = b(d_i) \circ b(d_j) \circ s_{ij}, v_j \in \mathcal{N}_i^k, \quad (24)$$

where $b(x) = \lfloor \log(x) + 1 \rfloor$ is a logarithmically binning function. The indicator approximating Weisfeiler-Lehman subtree kernel is defined as

$$\phi_{WL}^i(v_j) = b(\mathbf{1}^{<i,j>}) \circ b(\mathbf{1}^{<i,j>}) \circ s_{ij}, v_j \in \mathcal{N}_i^k, \quad (25)$$

where $\mathbf{1}^{<i,j>}$ is a vector of length $k+1$ whose l th element is the count of v_j 's neighbors at distance l to v_i in $\mathcal{G}_i^k$, i.e.

$$\mathbf{1}_l^{<i,j>} = |\{v_{j'} \in \mathcal{N}_j | s_{ij'} = l\}|, l = 0, \dots, k. \quad (26)$$

After subgraph relabeling, the random walks starting from v_i are performed on each $\mathcal{G}_i^k$. The nodes are relabeled indicated by Eqs. (24) or (25) while only v_i is not relabeled. And embeddings are learned via SGNS on the generated sequences.

NODE2BITS [76]. NODE2BITS is designed for entity resolution on temporal networks. Here we use τ_{ij} to denote the timestamp of edge e_{ij} . To integrate temporal information, NODE2BITS utilizes temporal random walks in which edges are sampled with non-decreasing timestamps. The following stepping probability is defined to capture short-term transitions in temporal walks

$$p_s(v_i, v_j) = \frac{\exp(-\tau_{ij}/T)}{\sum_{(v_i, v_{j'}) \in \mathcal{E}} \exp(-\tau_{ij'}/T)}, \quad (27)$$

where T is the maximal duration between all timestamps. The stepping probability in long-term policy is defined

similarly with positive signs. Multiple walks are generated for each edge and the temporal context of different hops Δt for a node can be extracted from the walks. Then structural features and attributes are fused in temporal walks. For each node v_i with a specific Δt , histograms are applied on multi-dimensional features (and node types if the network is heterogeneous) to aggregate information in the neighborhood, and they are concatenated as a vector $(\mathbf{H}_{HIST})_i^{\Delta t}$. SimHash [75] is applied by projecting the histogram $(\mathbf{H}_{HIST})_i^{\Delta t}$ to several random hyperplanes for generating binary hash-code $(H_{HASH})_i^{\Delta t}$. The final embeddings are obtained via concatenation on $(H_{HASH})_i^{\Delta t}$ across different Δt s.

Remark. The above three methods are very different on their motivations of utilizing structural feature-based random walks. Role2vec assigns roles first and then employs random walks with role indicators. It essentially captures proximity between assigned roles. RiWalk relabels the walks in subgraphs to approximate graph kernels. NODE2BITS uses random walks as neighbor feature aggregators.

3.3 Deep Learning Models

Recently, a few works focus on leveraging deep learning techniques to role-oriented network representation learning. Though deep learning can provide more varied and powerful mapping mechanisms, it needs to be trained with more carefully designed structural information guidance.

3.3.1 Structural Information Reconstruction/Guidance

DRNE [51]. DRNE is proposed to capture regular equivalence in networks, so it learns node embeddings in a recursive way with the following loss function

$$\mathcal{L}_{equiv} = \sum_{v_i \in \mathcal{V}} \|\mathbf{H}_i - \check{\mathbf{H}}_i\|_2^2, \quad (28)$$

where $\check{\mathbf{H}}_i$ is the aggregation of the neighbors' embeddings via a layer normalized Long Short-Term Memory. To make the neighbor information available for LNLSTM, for each node v_i , it downsamples a fixed number of neighbors with large degrees and orders them based on the degrees. Denoting their embeddings as $\{\mathbf{H}_{(1)}, \dots, \mathbf{H}_{(T)}\}$ the aggregating process is $\check{\mathbf{H}}_{(t)} = \text{LNLSTM}(\mathbf{H}_{(t)}, \check{\mathbf{H}}_{(t-1)})$ and finally $\check{\mathbf{H}}_i = \check{\mathbf{H}}_{(T)}$.

Additionally, DRNE proposes a degree-guided regularizer to avoid the trivial solution where all embeddings are $\mathbf{0}$. The regularizer is as follows:

$$\mathcal{L}_{deg} = \sum_{v_i \in \mathcal{V}} (\log(d_i + 1) - \text{MLP}_{deg}(\check{\mathbf{H}}_i))^2. \quad (29)$$

The regularizer with a parameter γ_{deg} is weighed and the whole model is trained via the combined loss

$$\mathcal{L} = \mathcal{L}_{equiv} + \gamma_{deg} \mathcal{L}_{deg}. \quad (30)$$

GAS [79]. Graph Neural Networks have the power to capture structure as they are closely related to Weisfeiler-Lehman (WL) test in some ways [80]. GAS applies a L -layer graph convolutional encoder, in which each layer is

$$\mathbf{H}^{(l)} = \sigma(\tilde{\mathbf{A}} \mathbf{H}^{(l-1)} \Theta^{(l-1)}), \quad (31)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\Theta^{(l-1)}$ is the parameter matrix in the l th layer. The input $\mathbf{H}^{(0)}$ could be $\tilde{\mathbf{A}}$ or an embedding lookup table. Here the sum-pooling propagation rule is applied instead of the original GCN [81] to better distinguish local structures. In fact, more powerful GNNs such as Graph Isomorphic Network [80] may further improve the performance. The key idea for GAS is using a few critical structural features as the guidance information to train the model. The features are extracted in a similar as proposed in ReFeX, but aggregated only once, normalized and not binned. With a MLP model as the decoder to approximate the features, i.e., $\hat{\mathbf{F}} = \text{MLP}_{dec}(\mathbf{H})$. The loss function is

$$\mathcal{L} = \|\mathbf{F} - \hat{\mathbf{F}}\|_F^2. \quad (32)$$

*RES*D [82]. *RES*D also adopts ReFeX [60] to extract appropriate features $\mathbf{F}_{ReFeX}$. It uses a Variational Auto-Encoder [83] architecture to learn the low-noise and robust representations:

$$\begin{aligned} \mathbf{Z}_i &= \text{MLP}_{enc}(\mathbf{F}_i) \\ \mu_i &= \mathbf{W}_\mu \mathbf{Z}_i + \mathbf{b}_\mu \\ \log(\sigma_i) &= \mathbf{W}_\sigma \mathbf{Z}_i + \mathbf{b}_\sigma \\ \mathbf{H}_i &= \mu_i + \sigma_i \odot \epsilon, \epsilon \sim \text{Gaussian}(\mathbf{0}, \mathbf{I}) \\ \hat{\mathbf{F}}_i &= \text{MLP}_{dec}(\mathbf{H}_i). \end{aligned} \quad (33)$$

The VAE model is trained via feature reconstruction. A degree-guided regularizer Eq.(29) designed in DRNE [51] is introduced in *RES*D for preserving topological characteristics. The combined objective is as follows:

$$\mathcal{L} = \|\mathbf{F} - \hat{\mathbf{F}}\|_F^2 + \gamma_{deg} \mathcal{L}_{deg}. \quad (34)$$

GraLSP [84]. *GraLSP* is a GNN framework integrating local structural patterns that can be employed on role-oriented tasks. For a node v_i , it captures structural patterns by generating w random walks starting from v_i with length l_w : $\mathcal{W}_i = \{\omega_{i1}, \dots, \omega_{iw}\}$, and then anonymizes them [85]. Each anonymous walk $\text{aw}(\omega)$ is represented as an embedding lookup table $\mathbf{u}_{\text{aw}(\omega)}$. Then the aggregation of neighborhood representation is designed as follows:

$$\begin{aligned} (\mathbf{H}_{nei})_i^{(l)} &= \text{MEAN}_{\omega \in \mathcal{W}_i, j \in \left[1, \left\lfloor \frac{2l_w}{|\omega|} \right\rfloor\right]} (\alpha_{i,\omega}^{(l)} (\mathbf{a}_{i,\omega}^{(l)} \odot \mathbf{H}_{\omega_j}^{(l-1)})) \\ \mathbf{H}_i^{(l)} &= \text{ReLU}(\mathbf{W}_{self}^{(l)} \mathbf{H}_i^{(l-1)} + \mathbf{W}_{nei}^{(l)} (\mathbf{H}_{nei})_i^{(l)}), \end{aligned} \quad (35)$$

where $\mathbf{W}_{self}$ and $\mathbf{W}_{nei}$ are trainable parameter matrices. $\alpha_{i,\omega}^{(l)}$ is learned attention values based on their local structure, i.e., the embeddings of anonymous walks. $\mathbf{a}_{i,\omega}^{(l)}$, the amplification coefficients, is also linearly mapped from $\mathbf{u}_{\text{aw}(\omega)}$. After L aggregations, the embeddings are $\mathbf{H} = \mathbf{H}^{(L)}$.

To preserve proximities between nodes, the loss function in DeepWalk [71] is leveraged

$$\begin{aligned} \mathcal{L}_{prox} &= - \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{N}_i} (\log \sigma(\mathbf{H}_i \mathbf{H}_j^\top) \\ &\quad - \gamma_{neg} \mathbb{E}_{v_k \sim P_n(v)} [\log \sigma(\mathbf{H}_i \mathbf{H}_k^\top)]). \end{aligned} \quad (36)$$

To capture structural similarities between nodes, *GraLSP* designs the following loss

$$\begin{aligned} \mathcal{L}_{struc} &= - \sum_{v_i \in \mathcal{V}, \omega_j, \omega_k, \omega_s \in \mathcal{W}_i} \log \sigma(\mathbf{u}_j^\top \mathbf{u}_k - \mathbf{u}_k^\top \mathbf{u}_s), \\ s.t. \quad &\hat{p}(\omega_j | v_i) > \hat{p}(\omega_j | \mathcal{G}), \hat{p}(\omega_k | v_i) > \hat{p}(\omega_k | \mathcal{G}), \\ &\hat{p}(\omega_n | v_i) < \hat{p}(\omega_n | \mathcal{G}). \end{aligned} \quad (37)$$

where $\hat{p}(\cdot)$ is the empirical distribution of anonymous walks. The objective is to combine the two losses with a trade-off parameter γ_{struc}

$$\mathcal{L} = \mathcal{L}_{prox} + \gamma_{struc} \mathcal{L}_{struc}. \quad (38)$$

GCC [86]. *GCC* is a pre-train Graph Neural Network model, which represents a node as $\mathbf{H}_i$ by encoding its node-centric subgraph and tries to distinguish the similar subgraphs from the dissimilar ones. It aims to leverage several large-scale networks to train the ability of a GNN-based model to discriminate node substructures in an unsupervised manner. When the scale of the pre-trained datasets is large enough, *GCC* has the power to recognize local connective patterns of nodes, and the representations generated by it can measure structural similarities. While other GNN methods (such as [87], [88]) either concentrate on community and proximity, or train model in an supervised manner. So we introduce *GCC* as a role-oriented network embedding method. Specifically, for each node v_i in a network, *GCC* extracts its k -hop reachable neighborhood as $\mathcal{G}_i^k$, and leverages the Graph Isomorphic Network [80] as the encoder whose convolutional layer is

$$\mathbf{H}^{(l)} = \text{MLP}_{GIN}((\mathbf{A} + (1 + \epsilon) \cdot \mathbf{I}) \mathbf{H}^{(l-1)}), \quad (39)$$

where ϵ could be a learnable or fixed parameter and the input attributes of GIN is initialized as the eigenvectors of $\mathcal{G}_i^k$. A similar subgraph instance of v_i is induced based on the nodes in the random work with restart starting from v_i . K dissimilar instances are subgraphs induced in the same way but starting from other nodes which could be in other networks. The representations $\{\mathbf{x}_0, \dots, \mathbf{x}_K\}$ of these $K+1$ instances are generated via another GIN encoder. The representation of the similar instance is denoted as $\mathbf{x}^+ \in \{\mathbf{x}_0, \dots, \mathbf{x}_K\}$, and *GCC* is pre-trained by a contrastive learning method called InfoNCE [89]

$$\mathcal{L} = \sum_{v_i \in \mathcal{V}} -\log \frac{\exp(\mathbf{H}_i \mathbf{x}^+ / \iota)}{\sum_{j=0}^K \exp(\mathbf{H}_i \mathbf{x}_j / \iota)}, \quad (40)$$

where ι is a hyper-parameter.

Remark. These deep methods incorporate model traditional role-related concepts used in shallow methods with deep learning techniques to map structural information into non-linear latent representations. *DRNE* learns regular equivalence and reconstructs node degrees. *GAS* uses structural features to guide the training of graph convolutional networks. *RES*D combines variants of *DRNE* and *ReFeX* via a VAE architecture. *GraLSP* reconstructs the similarities based on anonymous walks. And *GCC* is a pre-train model which encodes node-centric subgraphs and is pre-trained via a constrastive learning manner.

TABLE 3
Statistic of the Networks for Node Classification and Clustering

Dataset	# Nodes	# Edges	# Classes	Density(%)	Mean Degree	Average CC	Transitivity
Brazil	131	1,074	4	12.6130	16.3969	0.6364	0.4497
Europe	399	5,995	4	7.5503	30.0501	0.5393	0.3337
USA	1,190	13,599	4	1.9222	22.8555	0.5011	0.4263
Reality-call	6,809	7,697	3	0.0332	2.2608	0.0178	0.0024
Actor	7,779	26,733	4	0.0888	6.8917	0.0790	0.0156
Film	27,312	122,706	4	0.0329	8.9855	0.1180	0.0278

3.4 Time Complexity Analysis

In addition to the principle, i.e., how embedding methods capture structural similarity, time complexity is also a key property of embedding methods, since the size of real-world networks is always very large and continuously growing. Thus, we summarize and compare the theoretical time cost of reviewed role-oriented methods in a formal way as shown in Table 2.

Here we only consider the time cost with respect to some basic statistics of networks. Recall that n is the number of nodes, m is the number of edges, and d_{avg}/d_{max} is the average/maximal node degree. And we use the notation ϖ to represent the expressions of other parameters used in each method, since we only reviewed their key procedure while many parameters are not mentioned. We should notice that for some methods, different settings and variants (e.g., the graph kernels in SEGK and the indicators in RiWalk) may lead to different total complexity. And some approximation skills used in some methods may reduce the complexity. To make a pure comparison on efficiency, we only consider the condition where the method achieves the smallest time cost while ignoring how the method performs in the condition.

Overall, the feature-based methods including those factorizing feature matrix (Section 3.1.1) and those leveraging feature-based random walks (Section 3.2.2), and deep learning methods (Section 3.3) take a complexity linear to the node number or edge number of networks. This is because both the two steps, structural property extraction and embedding, are node-wise or edge-wise. The time cost of similarity-based methods including those factorizing similarity matrix (Section 3.1.2) and those leveraging similarity-biased random walks (Section 3.2.1) is super-linear to the edge number or node number. This is because no matter what approximation is applied, pair-wise similarity need to be computed. Note that the comparison is only based on basic network statistics. The superiority relationship on efficiency between methods can not directly derived from Table 2 as the complexity that ϖ represents is unknown. For clearer understanding, please further refer to our empirical analysis in Section 4.3.

4 EXPERIMENTAL EVALUATION

In this section, we show the comprehensive analysis of these popular role-oriented embedding methods on widely used benchmarks. The experimental evaluations on both efficiency and effectiveness are conducted. To analyze the efficiencies of these methods, we compare their running time for generating node representations on both real-world and synthetic networks with varying sizes. To evaluate the effectiveness of

these methods, we select four tasks for the evaluation, including (1) the classification experiment based on the ground-truth labels of datasets by comparing the Micro-F1 and Macro-F1 scores, (2) the clustering experiment by comparing some clustering indices with K-Means model in an unsupervised manner. (3) the visualization experiment by plotting the node representations in a 2-D space to observe the relationships between node embeddings and their roles. (4) the top-k similarity search experiment to see if nodes in the same role are mapped into close position in the embedding space.

It's worth noting that we do not select link prediction for the experimental study, mainly because (1) only limited previous role-oriented NE methods evaluate the performance of this task (see Table 2). Thus, it is difficult to make a consistent and fair comparison for all these methods. (2) It has been demonstrated that global information, i.e., roles, is less useful than local information, i.e., proximity, in link prediction task [90]. So we believe that this task is more suitable for proximity-preserving rather than role-oriented node embeddings.

4.1 Datasets

We perform our experiments on the following datasets which form unweighted and undirected networks to illustrate the potential of role-oriented network embedding methods in capturing roles. Based on the type of conducted experiments, these networks are divided into two groups. One group of networks, including *Brazil*, *Europe*, *USA*, *Reality-call*, *Actor* and *Film*, are for node classification and clustering in which the nodes are labeled based on some role-related rules. The other networks are all Wiki-talk networks in which some nodes have the same but very rare role. These Wiki-talk networks are for top-k similarity search. We show some statistical characteristics from various aspects of these networks in Tables 3 and 4, respectively. More details about the datasets can be found in the Supplementary Materials, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TBDATA.2021.3131610>.

4.2 Experimental Settings

Although different methods with different embedding mechanisms have been proposed, according to our proposed two-level classification taxonomy discussed in Section 2, for each class of role-oriented network embedding, we choose 4 methods to analyze the performance on different role related tasks. In specific, RoIX [56], RIDeRs [62], GraphWave [63] and SEGK [58] belong to the matrix factorization category. Struc2vec [50], struc2gauss [77], Role2Vec [73] and NODE2BITS [76]

TABLE 4
Statistic of the Networks for Top-k Similarity Search

Dataset	# Nodes	# Edges	# Bots	# Admins	Density(%)	Mean Degree	Average CC	Transitivity
ht-wiki-talk	446	758	24	0	0.7638	3.3991	0.0941	0.0085
br-wiki-talk	1,049	2,330	35	8	0.4239	4.4423	0.1998	0.0410
cy-wiki-talk	2,101	3,610	31	16	0.1636	3.4365	0.1579	0.0090
oc-wiki-talk	3,064	4,098	43	4	0.0873	2.6749	0.0994	0.0023
eo-wiki-talk	7,288	14,266	120	21	0.0537	3.9149	0.1206	0.0085
gl-wiki-talk	7,935	19,887	12	14	0.6318	5.0125	0.4913	0.0037

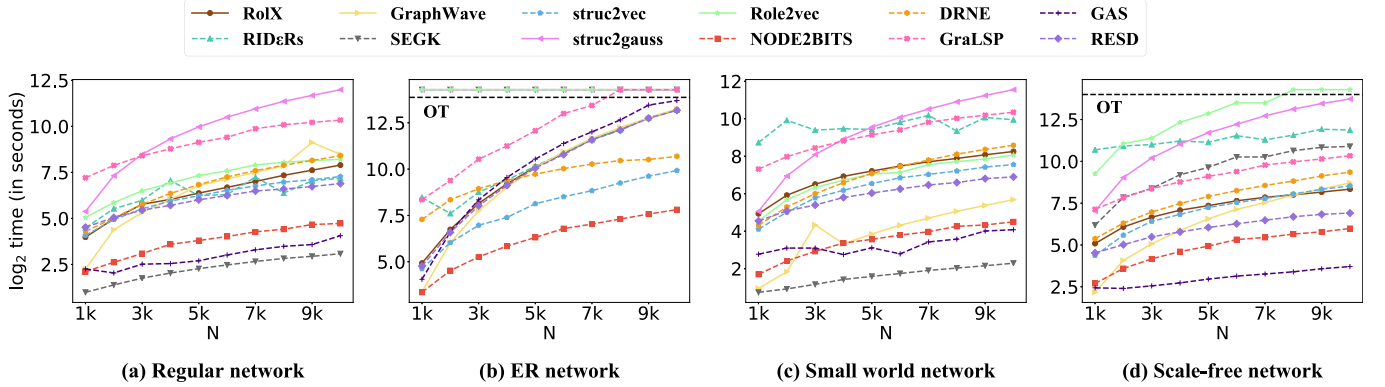


Fig. 4. Running time of 12 methods on four different types of synthetic networks with different sizes.

are all based on random walk. DRNE [51], GraLSP [84], GAS [79] and RESD [82] pertain to the scope of deep learning. In the subsequent experiments, all the parameters are fine-tuned. Note that for Role2vec, we use motif-count features as the default setting, but it causes the out-of-memory issue in large networks so we use node degrees to circumvent. The datasets and source code used in the experiments are available at <https://github.com/cspjjiao/RONE>.

4.3 Efficiency Analysis

All experiments are performed on a machine with Intel(R) Xeon(R) CPU E5-2680 v4 at 2.40GHz and 125GB RAM. In this experiment, by ignoring the differences in some implementation details, we report the running time of these methods on the above six real networks. The results are shown in Fig. 5. The y -axis represents the average logarithmic time (seconds) of 10 times running of each method on one specific network and the x -axis is the size (number of nodes) of the network. If the value is above the dotted line, it means that the cost of the corresponding method is beyond our tolerance. Overall, the methods based on deep learning are relatively efficient compared to others, especially for the methods that need higher-order features. GAS [79], RESD [82] and NODE2BITS [76] are the three most efficient methods and RolX [56] also has competitive results. struc2gauss [77], SEGK [58] and Role2vec [73] cost more time to learn the node embeddings since they need to compute higher-order features, e.g., motif. In particular, as the most classical method proposed in the early stage, RolX still shows competitive in efficiency.

However, the number of nodes in these networks grows in an unknown way. In order to show the efficiency of different methods more fairly, we generate four different types of networks with linear variation of the size. The details are as

following. Regular network. It is a regular graph in which each node has the same number of neighbors and we set it as 3. ER network [91]. It is based the ER random graph and we set the probability of having a link between any two nodes as 0.1. Small world network. It comes from the popular Watts–Strogatz model [92] and for each link, we set its rewiring probability as 0.5. Scale-free network. It is based on the Barabási–Albert model [93] with preferential attachment, for each new nodes, the number of its links is set as 3.

For each type of network, we vary the number of nodes N from 1,000 to 10,000. The running time of the baselines is shown in Fig. 4. From these results, some conclusions can be drawn. First, the ER network is relatively dense for its fixed link probability. The average degree of nodes in larger networks is larger correspondingly. In this type of network, NODE2BITS has significant efficiency advantage than all others, and it also achieves competitive results in other three

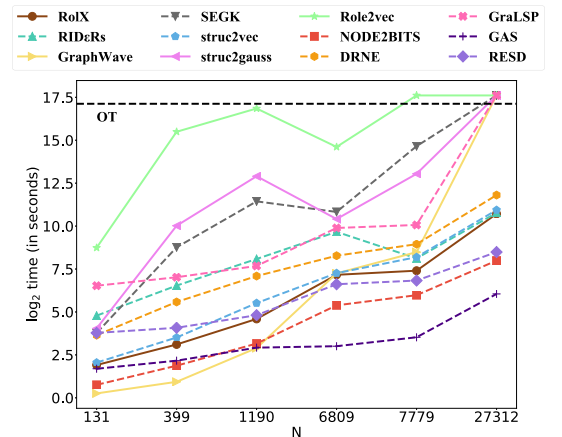


Fig. 5. Running time of 12 methods on the six real networks.

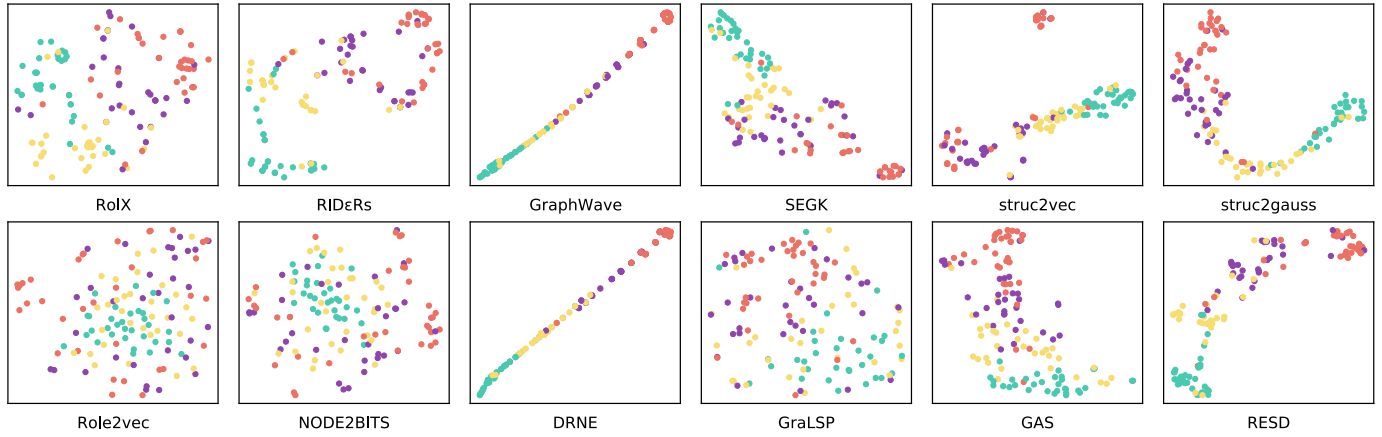


Fig. 6. Visualization results of 12 methods using t-SNE on the Air Brazil network. The points denote the nodes and the same color indicates that they belong to the same label. Two nodes are more closer to each other if their embeddings are role-similar.

TABLE 5
Node Classification Average F1 Score on Different Networks

Method	Brazil		Europe		USA		Reality-call		Actor		Film	
	Micro	Macro	Micro	Macro	Micro	Macro	Micro	Macro	Micro	Macro	Micro	Macro
RolX	0.749	0.741	0.556	0.546	0.623	0.617	0.593	0.383	0.465	0.451	0.494	0.396
RIDeRs	0.790	0.783	0.539	0.519	0.626	0.618	0.635	0.396	0.470	0.448	0.479	0.380
GraphWave	0.762	0.753	0.526	0.491	0.517	0.469	0.839	0.516	0.251	0.180	OM	OM
SEGK	0.723	0.718	0.536	0.524	0.615	0.607	0.839	0.514	0.479	0.460	OM	OM
struc2vec	0.766	0.757	0.577	0.572	0.599	0.594	0.593	0.376	0.463	0.456	0.474	0.365
struc2gauss	0.730	0.715	0.585*	0.580*	0.641	0.632	0.603	0.391	0.456	0.449	OT	OT
Role2vec	0.385	0.358	0.362	0.347	0.467	0.455	0.541	0.354	0.277†	0.277†	0.278†	0.243†
NODE2BITS	0.593	0.577	0.490	0.477	0.583	0.573	0.524	0.354	0.442	0.424	0.542*	0.428*
DRNE	0.716	0.700	0.542	0.521	0.601	0.588	0.630	0.478	0.462	0.449	0.453	0.339
GAS	0.757	0.750	0.583	0.574	0.668*	0.659*	0.841*	0.529*	0.480*	0.466*	0.490	0.380
RESL	0.797*	0.791*	0.557	0.544	0.640	0.634	0.607	0.411	0.476	0.463	0.516	0.415
GraLSP	0.510	0.490	0.455	0.422	0.535	0.523	0.454	0.300	0.341	0.316	OM	OM

types of networks. Second, SEGK is very efficient on the regular and small world networks but time-consuming on the ER and scale-free networks. Third, GAS is the most effective on the scale-free networks and has second results only to the SEGK. Besides, RolX and RESL rank in the middle of this experiments for they all are based on ReFeX [60] and reconstruct the features with different methods. GAS is one of the most efficient on almost all the networks. GAS reconstructs a small number of primary features which leads to few parameters and epochs for loss convergence. NODE2BITS consumes little time on all the networks stably. Because its feature aggregation process, which is based on random walk and the SimHash mapping process, is almost linear to the number of edges.

4.4 Visualization Analysis

Visualization can help to understand the performance of the different methods intuitively. It is also a common way to understand the network structure and the methods. In this section, we show the t-SNE results of these methods based on the Brazil network in Fig. 6. It can be observed that, RIDeRs [62], GraphWave [63], SEGK [58], struc2vec [50], struc2gauss [77], DRNE [51], GAS [79] and RESL [82] can be more accurate to identify the labels compared with others. The cyan nodes correspond to the airports with large passenger flow and all these methods can embed it effectively except NODE2BITS [76]. The red nodes represent

the marginal airports. These nodes are usually hard to identify because they are more scattered in the network. Therefore, these methods can not identify this kind of nodes well. Besides, although we expect to embed nodes with the same role into close points in latent space, the positions of nodes should be slightly different even if they are in the same role. From this view, struc2gauss, SEGK and GAS are better selections. GAS benefits from the low-dimensional features which lead to the same-role embeddings gathering. The embeddings of SEGK and struc2gauss shape some clumps since they are directly constrained by effective similarities.

4.5 Classification Results

Here we report the performance of the classification experiment on the six networks. In detail, with the learned node embeddings of each method, we randomly select 70% of the nodes as the training set and the others as the test set and take the logistic regression classifier to learn the labels of nodes. The results are shown in Table 5. It represents the Micro-F1 and Macro-F1 value of classification results. To avoid losing generality, we report the average results of 20 times of independent runs on each network.

As shown in Table 5, for each column, we mark the values of methods with significant advantages, i.e., the top results of these methods. OM and OT mean that it cannot be calculated for fixed memory and limited time, and the †

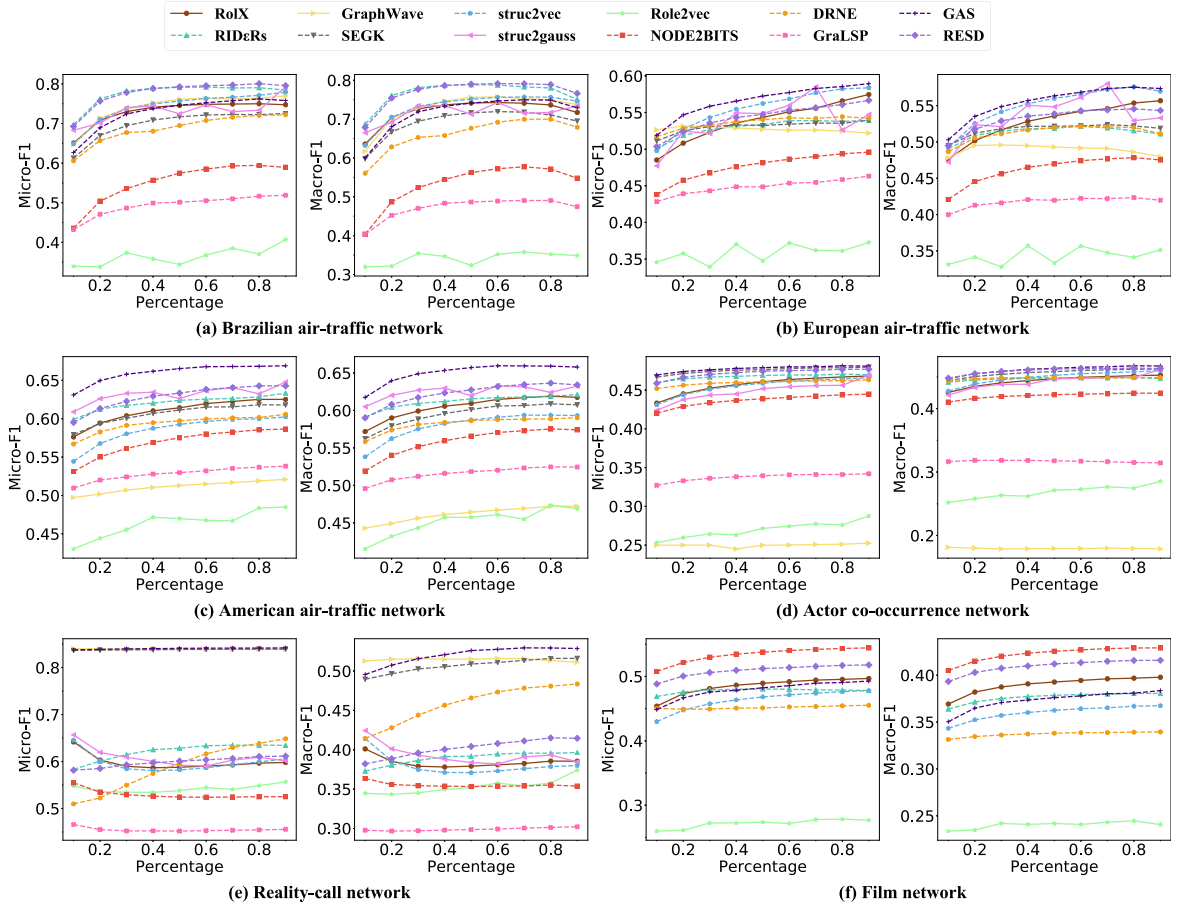


Fig. 7. Classification results of the 12 role-oriented network embedding methods on the six real-world networks. The performance is based on the Micro-F1 and Macro-F1 with different percentages of training set, and reported as the average values of 20 runs.

denotes that Role2vec uses degree features rather than motif features because it is very time-consuming.

For the three air-traffic networks, RESD [82] performs best on Brazil and RIDeRs [62] follows because the ReFeX [60] features used by them can effectively capture structural similarities in small networks. While on the other two networks, GAS [79] and struc2gauss [77] perform well. For the rest three large networks, struc2gauss cannot generate node embeddings within a limited time, though it performs well in small networks. GAS outperforms others on Reality-call and Actor, while NODE2BITS [76] gets the highest score on Film.

Among all the datasets, we observe that Role2vec [73] obtains the worst performance. This may be because it leverages the features-based random walk and damages the original network structure. It can be applied to link prediction task, but is not proper for role discovery. Though GraLSP [84] leverages the local structural patterns, it concentrates more on node proximity and fails to distinguish roles of nodes.

In Section 3, we have divided these methods into three types: random walk, matrix factorization, and deep learning. In general, deep learning methods perform well on all the networks. For the three small air-traffic networks, random walks (struc2vec [50] and struc2gauss) perform well, while matrix factorization methods get better results on large networks, even if some of them cannot obtain embeddings because of the out of memory error. Overall, deep

learning methods are scale-well and can effectively discover roles.

For matrix factorization methods, RIDeRs is better than RolX [56] except on Film, and they both can be applied to large-scale networks. GraphWave [63] and SEGK [58] perform best on Reality-call but consume too much memory. For random walk methods, struc2vec and struc2gauss show their superiority on small networks, while NODE2BITS is more suitable for large-scale networks. As for deep learning methods, GAS and RESD stay ahead of other approaches on all networks.

As shown in Fig. 5, for RESD, NODE2BITS, and GAS, there is a linear relationship between their computational complexities and network sizes. In Table 5, GAS and RESD perform well on all networks, while NODE2BITS only gets the best score on Film. Obviously, combining structural features and deep learning models has advantages in both time and classification.

Furthermore, in order to analyze the influence of different percentages of training set on classification results, we also show the performance of these methods with varying sizes of training set (from 10% to 90%). The results are shown in Fig. 7. The experimental settings are the same as introduced above.

In general, as the ratio of the training set increases, the F1 scores of classification results improved. On the three air-traffic networks, we observe a significant increase, but when we use too many training samples (more than 80%), the scores go

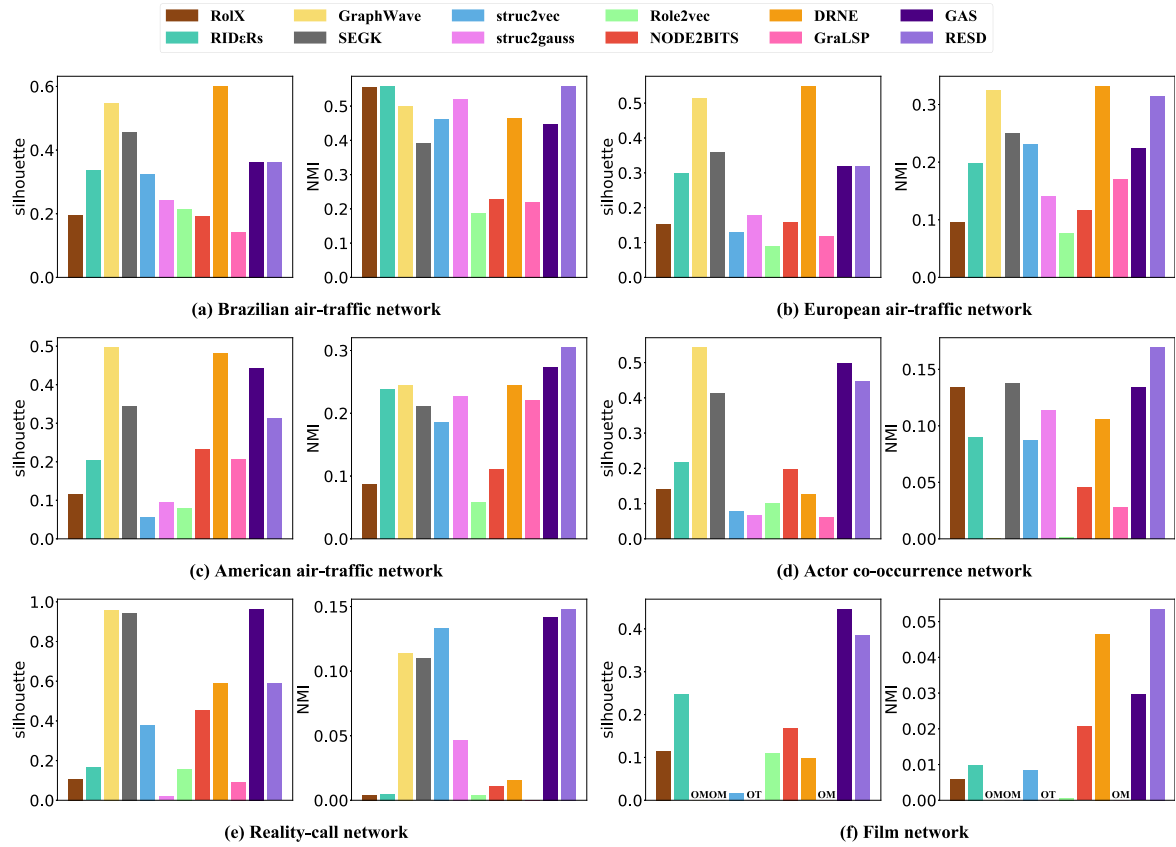


Fig. 8. Clustering results of the 12 role-oriented network embedding methods on the six real-world networks. The performance is based on the silhouette coefficient and NMI and evaluated on 10 times run.

down because of over-fitting. The scores on large networks are stable, because a small percentage of training samples are sufficient enough to make the linear model constraint.

The scores on Reality-call network are different to others, because the distribution of labels is unbalanced, and the Micro-F1 scores are higher than Macro-F1. In general, we observe the scores of Role2vec fluctuate, which means that Role2vec fails on the task of role classification. GraLSP also performs a little better than Role2vec, but it gets worse on Reality-call. Other methods show similar tendencies, while deep learning methods (GAS and RESD) and NODE2BITS exhibit significant superiority.

4.6 Clustering Results

In social science, the role discovery is usually denoted as an unsupervised task which can be implemented by unsupervised clustering in machine learning. In this experiment, we study the role clustering problem based on the embedding of different methods. Based on the embedding, we use the k-means algorithm to obtain the clusters, and report the silhouette coefficient [94] and Normalized Mutual Information (NMI) [95] in Fig. 8. The silhouette coefficient is a measure of how similar an object is to its cluster compared to other clusters. The result only relies on the embeddings themselves rather than true labels. While the NMI measures the similarity between clustering results and real distribution.

We observe that GraphWave [63] obtains high scores on both measurements on air-traffic networks, but in larger networks (Actor and Reality-call), it gets low NMI scores. DRNE [51] shows a similar tendency because its aggregation

methodology leads to obvious clustering, but may not fit the true distributions. The results of Role2vec [73] and GraLSP [84] are similar to classification, and they fail to cluster roles of nodes. The performances of other random walk and matrix factorization methods are moderate. In general, it can be observed that deep learning methods (GAS [79] and RESD [82]) show overall superiority. They perform well on small networks but significantly outperform others in large-scale networks. This is also consistent with findings of deep learning methods in other domains that those can better learn the patterns from larger-scale data.

4.7 Top-k Similarity Search

We conduct top-k similarity search experiments on the 6 Wiki-talk networks to evaluate the ability to retrieve rare roles of different methods. Specifically, after generating role-based embeddings for each node through a candidate method, we find the k most similar users for each bot or administrator by computing the euclidean distance between the embeddings. The mean precision for both bots and administrators on different sizes of the retrieved node list, i.e., different k s, is reported in the Supplementary Materials, available online.

In general, we assume that nodes literally having the same role (such as bots in Wiki-networks) in a network have similar structural patterns. However, there are more bots than administrators in most Wiki-talk networks. Almost every method achieves higher precision on searching administrators than that on searching bots when the retrieved list size k is fixed and smaller than or close to the

number of administrators. This shows that the structures of bots are more irregular or these methods can not effectively capture the specific structural patterns of bots. The structural representation learning on literal roles is much more difficult than on function-based defined roles (such as the classes in Table 3), since the latent patterns of literal roles are usually not reflected in the plain structural properties.

What's more, the inconsistency between literal roles and node structure is obvious across networks. We can come to this conclusion gradually through the following observations: the proportion of the same literal role in different networks varies greatly as shown in Table 4; none of the compared methods can produce top results on all the networks with k fixed; although the proportion of bots in cy-wiki-talk and oc-wiki-talk networks are very close, the results of almost every method on cy-wiki-talk are better than those on oc-wiki-talk. Thus, the literal roles in different networks are less comparable and transferable to those roles (such as the classes in Table 3) defined directly by node functions.

Though no one method achieves the best performance on all the networks, the ranks of each method on a network with a different retrieved list size k are usually close. Some methods, such as RIDeRs [62], GraphWave [63] and NODE2BITS [76] show their wider applicability across different networks. NODE2BITS surprisingly achieves good and stable results compared with its performance on node classification and clustering tasks. Its feature aggregation mechanism based on random walks makes it lose some elaborate information but capture more latent structural patterns. Similarly, RIDeRs and GraphWave can also capture more latent structural patterns via ε -equitable refinement and characterizing wavelet distributions, respectively. Role2vec still lacks competitiveness on this task, since it assigns roles based on features first and generates embeddings capturing proximities between roles as we argued above. The deep learning methods DRNE [51], GAS [79] and RESD [82] show mediocre performance as they overfit some features due to the deep process of feature reconstruction. While the similarity captured by GraLSP [84] based on anonymous walks is too coarse.

4.8 Discussion on Experiments

In this section, we show a variety of experiments on some popular role-oriented network embedding methods. These methods cover all the three major categories and the five minor categories we propose. We evaluate their effectiveness on classification, clustering, visualization and top- k similarity search and efficiency on networks at different scales.

We find that no one method can outperform the others on all tasks. As we argued above, each task has a different focus and the evaluated methods are only suitable for parts of them. On balance, GAS [79] and RESD [82] are good at visualization, node classification and clustering tasks. NODE2BITS [76] shows outstanding performance on both top- k similarity search and efficiency test. Role2vec [73] ceases to be effective on these role-based tasks as it assigns roles based on features directly while the embeddings do not capture the structural similarities. It specializes in link prediction for non-direct role-based tasks. Similarly, the

structural similarity objective of GraLSP [84] is not powerful enough for role-based tasks. The performance of the other methods fluctuates a lot on different tasks and networks. Thus, it is still an open problem how we can effectively and efficiently capture both fine and essential role information.

Though some deep learning methods seem good at many experiments, they have not yet addressed the nature of the role. The deep learning manners give them better capabilities on machine learning tasks such as classification and clustering. However, they utilize additional skills to extract structural information, and lack a suitable objective function to embed it without bias and impurities. Thus, deep role-based representation research, which is still in the beginning stage, is promising and needs more attention.

5 APPLICATIONS

Role discovery can complement community detection in network clustering and has always been the focus of social science and network science research. Role oriented network embedding has gradually become a focus of graph machine learning. It can benefit node ranking, community detection, information spreading and other problems in complex networks. It also contributes to the research and development of graph neural networks in machine learning. So we summarize some important applications of role oriented network embedding as follows.

First, it is beneficial for a variety of network mining tasks. Community detection and role discovery are two complementary tasks from the local and global perspective of network structures respectively [96]. Thus, combining these two tasks can help to achieve better network generation and mutually improve each other. For instance, MMCR has been proposed to integrate community detection and role discovery into a unified model and detect both of them simultaneously for information networks [97]. The proposed method extends the Mixed Membership Stochastic Blockmodel (MMSB) [35] to combine the generative process of both community and role. REACT [52] analyzes the community structure and role discovery under a unified framework and describes their relations via non-negative matrix factorization. Moreover, role-oriented embedding can also help other tasks, including link prediction [90], anomaly detection [56], and structural similarity search on network [49].

Second, it can help to analyze the network dynamics and evolution. As we know, nodes with different roles may have an inequable influence on information diffusion. RAIN [98] analyzes the effect of users with different roles on their reposting messages and models the generation of diffusion process under a unified probabilistic framework. Then, with a general representation learning on the network, it can effectively improve the performance of cascade [99] and its popularity [100] prediction. Besides, analyzing the roles of nodes could be used to predict the network evolution and its dynamic behaviors [101], [102] and detecting the varying roles of nodes could capture the dynamic structural information [103].

Moreover, it has induced some new graph neural networks with more expressive ability. Although GNNs with message passing are mainly used for supervised node or graph classification and link prediction, some new

architectures have been developed to improve their expressive ability with the heterogeneity and roles in the network. Geom-GCN [104] has been proposed to use the role embedding method to obtain the structural neighborhood and a bi-level aggregation in GNN. ID-GNN [87] extracts the ego network centered at one node of the network and takes rounds of heterogeneous message passing. It can compute the shortest path and clustering coefficient of the network. These features are extremely relevant to the role. GCC [86] also makes use of ego network and then employs pre-training for role-oriented network embedding.

Besides, it can shed light on the discovery of new patterns in specific networks. Role discovery and identification can be used to predict social behaviors, identities and temporal patterns [105]. [106] analyzes the user behaviors in the Blog social network and learns roles from the concepts of activity, influence and competition. It also can predict the sentiment and topics with the proposed definitions of roles. EMBER [68] considers professional role inference in a large-scale email network, so it can capture and distinguish the behavior similarity of nodes automatically. SADE [107] combines role information and subgraph embedding to detect subgraph-level anomalies from financial transaction networks.

6 FUTURE DIRECTIONS

Although many creative and innovative methods have been proposed for role oriented network embedding, it still faces some problems and challenges to be solved in this field. In this section, we will discuss these challenges as future directions.

Role-Oriented Embedding on Dynamic Networks. Most methods reviewed in this survey are for static networks. However, real-world networks are naturally dynamic and continuously stream over time with evolving structures. Thus, role-oriented NE methods for dynamic networks are of fundamentally practical and theoretical importance. There have only been a few approaches to role discovery on dynamic networks, e.g., Role-Dynamics [108] and DyNMF [103]. However, some questions remain open. For example, how to separate dynamic networks into different snapshots and how to model the deletion of nodes and edges. Thus, additional investigation is needed to extend current NE methods for dynamic scenarios.

Other Types of Embedding Spaces. Besides the euclidean space, a variety of methods project the nodes into the Hyperbolic space [109] in proximity preserved network representation learning. For role oriented embedding, as we know, the only work is Hyperboloid [110] which extends the struc2vec [50] with the hyperboloid model. Considering the non-euclidean space is quite suitable for network embedding because of the power-law distribution of network data, it is interesting and meaningful to have some in-depth analysis and understanding of utilizing hyperbolic neural networks for this problem.

Construction of Larger-Scale Benchmarks. All the methods for role oriented network embedding are evaluated on relatively small-scale network data with thousands of nodes (seen in Section 4). However, real-world networks are often of a massive scale, e.g., there are billions of users in social networks. Constructing larger-scale benchmark datasets is very imp-

ortant to evaluate existing approaches in effectiveness, efficiency and robustness, and also beneficial for researchers to develop new models.

Interpretation on Roles and Role-Oriented Embeddings. In social science, roles often correspond to social identities, e.g., students and teachers in a school. Real-world networks may not contain such information, so it is difficult to understand the discovered roles. The lack of interpretability of roles and role-oriented embeddings may significantly impact our ability to gain insights into these roles. However, so far, there has not been much work focusing on discovering roles that are interpretable. One exception is RolX [56], and it attempts to explain roles based on several structural measures such as centrality. More research is needed to discover roles and learn embeddings that are more interpretable.

Deep Theoretical Analysis. Important but not the final, network embedding for role discovery lacks theoretical analysis. Although some methods stem from a certain type of equivalence relation, the theoretical analysis has been ignored. Moreover, current methods cannot capture universal node representations and provide the upper/lower bound of the expressive ability. It is expected to have solid theoretical analysis in this field similar to that on graph neural networks [80].

7 CONCLUSION

Role-oriented network embedding can complement the category of network representation learning. It has gradually become one of the most important research focuses on network embedding. In this survey, we have proposed a general understanding mechanism for role-oriented network embedding approaches and a two-level classification ontology based on different embedding principles. Using this ontology, we categorize a series of popular methods into different groups. Then, we review the principles and innovations of selected representative embedding methods in different categories. We further conduct comprehensive experiments to evaluate these representative methods, including role classification and clustering, top-k similarity search, visualization and their efficiency test. Last, we summarize the important applications of this problem and outline some future directions. We believe that this survey can help to understand and deepen role-oriented network embedding, and it will attract more attention from network science and deep learning.

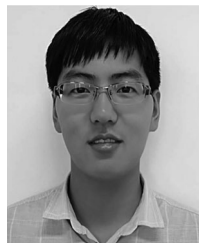
REFERENCES

- [1] S. H. Strogatz, "Exploring complex networks," *Nature*, vol. 410, no. 6825, pp. 268–276, 2001.
- [2] J. Gao, B. Barzel, and A.-L. Barabási, "Universal resilience patterns in complex networks," *Nature*, vol. 530, no. 7590, pp. 307–312, 2016.
- [3] V. Martínez, F. Berzal, and J.-C. Cubero, "A survey of link prediction in complex networks," *ACM Comput. Surv.*, vol. 49, no. 4, pp. 1–33, 2016.
- [4] Y. Liu, T. Safavi, A. Dighe, and D. Koutra, "Graph summarization methods and applications: A survey," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 1–34, 2018.
- [5] Z.-K. Zhang, C. Liu, X.-X. Zhan, X. Lu, C.-X. Zhang, and Y.-C. Zhang, "Dynamics of information diffusion and its applications on complex networks," *Phys. Rep.*, vol. 651, pp. 1–34, 2016.
- [6] M. Á. Serrano and M. Boguna, "Clustering in complex networks. I. General formalism," *Phys. Rev. E*, vol. 74, no. 5, 2006, Art. no. 056114.

- [7] M. Girvan and M. E. Newman, "Community structure in social and biological networks," in *Proc. Nat. Acad. Sci.*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [8] M. Rosvall and C. T. Bergstrom, "An information-theoretic framework for resolving community structure in complex networks," in *Proc. Nat. Acad. Sci.*, vol. 104, no. 18, pp. 7327–7331, 2007.
- [9] J. Cheng, M. Chen, M. Zhou, S. Gao, C. Liu, and C. Liu, "Overlapping community change-point detection in an evolving network," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 189–200, Mar. 2020.
- [10] R. A. Rossi and N. K. Ahmed, "Role discovery in networks," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 4, pp. 1112–1131, Apr. 2015.
- [11] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, no. 3–5, pp. 75–174, 2010.
- [12] R. K. Merton and R. C. Merton, *Social Theory and Social Structure*. New York, NY, USA: Simon and Schuster, 1968.
- [13] G. Liu, Y. Wang, and M. Orgun, "Social context-aware trust network discovery in complex contextual social networks," in *Proc. AAAI Conf. Artif. Intell.*, 2012, pp. 101–107.
- [14] S. Gilpin, T. Eliassi-Rad, and I. Davidson, "Guided learning for role discovery (GLRD) framework, algorithms, and applications," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2013, pp. 113–121.
- [15] M. Chen, K. Kuzmin, and B. K. Szymanski, "Community detection via maximization of modularity and its variants," *IEEE Trans. Comput. Social Syst.*, vol. 1, no. 1, pp. 46–65, Mar. 2014.
- [16] X. Li, M. K. Ng, and Y. Ye, "Multicomm: Finding community structure in multi-dimensional networks," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 4, pp. 929–941, Apr. 2014.
- [17] N. Stanley, S. Shai, D. Taylor, and P. J. Mucha, "Clustering network layers with the strata multilayer stochastic block model," *IEEE Trans. Netw. Sci. Eng.*, vol. 3, no. 2, pp. 95–105, Apr.–Jun. 2016.
- [18] P. Chopade and J. Zhan, "A framework for community detection in large networks using game-theoretic modeling," *IEEE Trans. Big Data*, vol. 3, no. 3, pp. 276–288, Sep. 2017.
- [19] L. Yang, X. Cao, D. Jin, X. Wang, and D. Meng, "A unified semi-supervised community detection framework using latent space graph regularization," *IEEE Trans. Cybern.*, vol. 45, no. 11, pp. 2585–2598, Nov. 2015.
- [20] Y. Pei, N. Chakraborty, and K. Sycara, "Nonnegative matrix tri-factorization with graph regularization for community detection in social networks," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, 2015, pp. 2083–2089.
- [21] X. Ma and D. Dong, "Evolutionary nonnegative matrix factorization algorithms for community detection in dynamic networks," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 5, pp. 1045–1058, May 2017.
- [22] X. Ma, D. Dong, and Q. Wang, "Community detection in multi-layer networks using joint nonnegative matrix factorization," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 2, pp. 273–286, Feb. 2019.
- [23] C. Tu *et al.*, "A unified framework for community detection and network representation learning," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 6, pp. 1051–1065, Jun. 2019.
- [24] F. Liu *et al.*, "Deep learning for community detection: Progress, challenges and opportunities," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 4981–4987.
- [25] H. Feng, J. Tian, H. J. Wang, and M. Li, "Personalized recommendations based on time-weighted overlapping community detection," *Inf. Manage.*, vol. 52, no. 7, pp. 789–800, 2015.
- [26] J. Zheng, S. Wang, D. Li, and B. Zhang, "Personalized recommendation based on hierarchical interest overlapping community," *Inf. Sci.*, vol. 479, pp. 55–75, 2019.
- [27] Z. Lu, X. Sun, Y. Wen, G. Cao, and T. La Porta, "Algorithms and applications for community detection in weighted networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 11, pp. 2916–2926, Nov. 2015.
- [28] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *Phys. Rep.*, vol. 659, pp. 1–44, 2016.
- [29] D. Jin *et al.*, "A survey of community detection approaches: From statistical modeling to deep learning," *IEEE Trans. Knowl. Data Eng.*, to be published, doi: [10.1109/TKDE.2021.3104155](https://doi.org/10.1109/TKDE.2021.3104155).
- [30] F. Lorrain and H. C. White, "Structural equivalence of individuals in social networks," *J. Math. Sociol.*, vol. 1, no. 1, pp. 49–80, 1971.
- [31] D. R. White and K. P. Reitz, "Graph and semigroup homomorphisms on networks of relations," *Social Netw.*, vol. 5, no. 2, pp. 193–234, 1983.
- [32] P. W. Holland and S. Leinhardt, "An exponential family of probability distributions for directed graphs," *J. Amer. Statist. Assoc.*, vol. 76, no. 373, pp. 33–50, 1981.
- [33] K. Nowicki and T. A. B. Snijders, "Estimation and prediction for stochastic blockstructures," *J. Amer. Statist. Assoc.*, vol. 96, no. 455, pp. 1077–1087, 2001.
- [34] K. Faust and S. Wasserman, "Blockmodels: Interpretation and evaluation," *Social Netw.*, vol. 14, no. 1–2, pp. 5–61, 1992.
- [35] E. M. Airoldi, D. Blei, S. Fienberg, and E. Xing, "Mixed membership stochastic blockmodels," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 33–40.
- [36] A. Arockiasamy, A. Gionis, and N. Tatti, "A combinatorial approach to role discovery," in *Proc. IEEE 16th Int. Conf. Data Mining*, 2016, pp. 787–792.
- [37] J. Qiu *et al.*, "NetSMF: Large-scale network embedding as sparse matrix factorization," in *Proc. World Wide Web*, 2019, pp. 1509–1520.
- [38] T. N. Kipf and M. Welling, "Variational graph auto-encoders," in *Proc. Neural Inf. Process. Syst. Bayesian Deep Learn. Workshops*, 2016, pp. 1–3.
- [39] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [40] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [41] P. Cui, X. Wang, J. Pei, and W. Zhu, "A survey on network embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 5, pp. 833–852, May 2019.
- [42] D. Zhang, J. Yin, X. Zhu, and C. Zhang, "Network representation learning: A survey," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 3–28, Mar. 2020.
- [43] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, early access, Mar. 17, 2020, doi: [10.1109/TKDE.2020.2981333](https://doi.org/10.1109/TKDE.2020.2981333).
- [44] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [45] P. Goyal and E. Ferrara, "Graph embedding techniques, applications, and performance: A survey," *Knowl.-Based Syst.*, vol. 151, pp. 78–94, 2018.
- [46] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [47] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Eng. Bull.*, vol. 40, no. 3, pp. 52–74, Sep. 2017.
- [48] H. Chen, B. Perozzi, R. Al-Rfou, and S. Skiena, "A tutorial on network embeddings," *arXiv preprint arXiv:1808.02590*, 2018.
- [49] R. A. Rossi, D. Jin, S. Kim, N. K. Ahmed, D. Koutra, and J. B. Lee, "On proximity and structural role-based embeddings in networks: Misconceptions, techniques, and applications," *ACM Trans. Knowl. Discov. Data*, vol. 5, no. 14, pp. 1–37, 2020.
- [50] L. F. Ribeiro, P. H. Saverese, and D. R. Figueiredo, "struc2vec: Learning node representations from structural identity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 385–394.
- [51] K. Tu, P. Cui, X. Wang, P. S. Yu, and W. Zhu, "Deep recursive network embedding with regular equivalence," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2357–2366.
- [52] Y. Pei, G. Fletcher, and M. Pechenizkiy, "Joint role and community detection in networks via l2, 1 norm regularized nonnegative matrix tri-factorization," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2019, pp. 168–175.
- [53] J. Jin, M. Heimann, D. Jin, and D. Koutra, "Towards understanding and evaluating structural node embeddings," *ACM Trans. Knowl. Discov. Data*, vol. 16, no. 3, pp. 1–32, Nov. 2021.
- [54] P. W. Holland and S. Leinhardt, "An exponential family of probability distributions for directed graphs," *J. Amer. Statist. Assoc.*, vol. 76, no. 373, pp. 33–50, 1981.
- [55] D. R. White and K. P. Reitz, "Graph and semigroup homomorphisms on networks of relations," *Social Netw.*, vol. 5, no. 2, pp. 193–234, 1983.
- [56] K. Henderson, *et al.*, "Rolx: structural role extraction & mining in large graphs," in *Proc. 18th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2012, pp. 1231–1239.

- [57] J. Guo, L. Xu, and J. Liu, "Spine: Structural identity preserved inductive network embedding," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2399–2405.
- [58] G. Nikolentzos and M. Vazirgiannis, "Learning structural node representations using graph kernels," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 5, pp. 2045–2056, May 2021.
- [59] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [60] K. Henderson et al., "It's who you know: Graph mining using recursive structural features," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2011, pp. 663–671.
- [61] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [62] P. V. Gupta, B. Ravindran, and S. Parthasarathy, "Role discovery in graphs using global features: algorithms, applications and a novel evaluation strategy," in *Proc. IEEE 33rd Int. Conf. Data Eng.*, 2017, pp. 771–782.
- [63] C. Donnat, M. Zitnik, D. Hallac, and J. Leskovec, "Learning structural node embeddings via diffusion wavelets," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1320–1329.
- [64] D. I. Shuman, P. Vandergheynst, and P. Frossard, "Chebyshev polynomial approximation for distributed signal processing," in *Proc. Int. Conf. Distrib. Comput. Sensor Syst. Workshops*, 2011, pp. 1–8.
- [65] R. A. Rossi, N. K. Ahmed, E. Koh, S. Kim, A. Rao, and Y. Abbasi-Yadkori, "A structural graph representation learning framework," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 483–491.
- [66] M. Heimann, H. Shen, T. Safavi, and D. Koutra, "Regal: Representation learning-based graph alignment," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage.*, 2018, pp. 117–126.
- [67] P. Drineas and M. W. Mahoney, "On the Nyström method for approximating a gram matrix for improved kernel-based learning," *J. Mach. Learn. Res.*, vol. 6, no. 12, pp. 2153–2175, 2005.
- [68] D. Jin et al., "Smart roles: Inferring professional roles in email networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2923–2933.
- [69] C. K. Williams and M. Seeger, "Using the nyström method to speed up kernel machines," in *Proc. Adv. Neural Inf. Process. Syst.*, 2001, pp. 682–688.
- [70] R. Jin, V. E. Lee, and H. Hong, "Axiomatic ranking of network role similarity," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2011, pp. 922–930.
- [71] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [72] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [73] N. K. Ahmed et al., "Role-based graph embeddings," *IEEE Trans. Knowl. Data Eng.*, early access, Jul. 2, 2020, doi: 10.1109/TKDE.2020.3006475.
- [74] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
- [75] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th Annu. ACM Symp. Theory Comput.*, 2002, pp. 380–388.
- [76] D. Jin, M. Heimann, R. A. Rossi, and D. Koutra, "Node2bits: Compact time- and attribute-aware node representations for user stitching," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2019, pp. 483–506.
- [77] Y. Pei, X. Du, J. Zhang, G. Fletcher, and M. Pechenizkiy, "struc2gauss: Structural role preserving network embedding via gaussian embedding," *Data Mining Knowl. Discov.*, vol. 34, no. 4, pp. 1072–1103, 2020.
- [78] M. Xuwei, G. Qin, Z. Qiu, M. Zheng, and Z. Wang, "Riwalk: Fast structural node embedding via role identification," in *Proc. IEEE Int. Conf. Data Mining*, 2019, pp. 478–487.
- [79] X. Guo, W. Zhang, W. Wang, Y. Yu, Y. Wang, and P. Jiao, "Role-oriented graph auto-encoder guided by structural information," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2020, pp. 466–481.
- [80] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–17.
- [81] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [82] W. Zhang, X. Guo, W. Wang, Q. Tian, L. Pan, and P. Jiao, "Role-based network embedding via structural features reconstruction with degree-regularized constraint," *Knowl.-Based Syst.*, vol. 218, 2021, Art. no. 106872.
- [83] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Representations*, 2014, pp. 1–14.
- [84] Y. Jin, G. Song, and C. Shi, "GralSP: Graph neural networks with local structural patterns," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 4361–4368.
- [85] S. Ivanov and E. Burnaev, "Anonymous walk embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2186–2195.
- [86] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1150–1160.
- [87] J. You, J. Gomes-Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 10737–10745.
- [88] J. You, R. Ying, and J. Leskovec, "Position-aware graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7134–7143.
- [89] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2019, *arXiv: 1807.03748*.
- [90] T. Lyu, Y. Zhang, and Y. Zhang, "Enhancing the network embedding quality with structural similarity," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2017, pp. 147–156.
- [91] P. Erdos and A. Renyi, "On random graphs i," *Publicationes Mathematicae Debrecen*, vol. 6, pp. 290–297, 1959.
- [92] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998.
- [93] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [94] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.
- [95] L. Danon, A. Diaz-Guilera, J. Duch, and A. Arenas, "Comparing community structure identification," *J. Statist. Mech.: Theory Exp.*, vol. 2005, no. 9, p. P09008, 2005.
- [96] Y. Pei, *On Local and Global Graph Structure Mining*, ser. SIKS Dissertation Series. Technische Universiteit Eindhoven, Eindhoven, The Netherlands, 2020.
- [97] T. Chen, L.-A. Tang, Y. Sun, Z. Chen, H. Chen, and G. Jiang, "Integrating community and role detection in information networks," in *Proc. SIAM Int. Conf. Data Mining*, 2016, pp. 72–80.
- [98] Y. Yang et al., "RAIN: Social role-aware information diffusion," in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 367–373.
- [99] J. Caverlee et al., "Inf-VAE: A variational autoencoder framework to integrate homophily and influence in diffusion prediction," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 510–518.
- [100] F. Zhou, X. Xu, K. Zhang, G. Trajcevski, and T. Zhong, "Variational information diffusion for probabilistic cascades prediction," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2020, pp. 1618–1627.
- [101] S. Jiang and H. Chen, "Natergm: A model for examining the role of nodal attributes in dynamic social media networks," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 3, pp. 729–740, Mar. 2016.
- [102] "Exploring the transition behavior of nodes in temporal networks based on dynamic community detection," *Future Gener. Comput. Syst.*, vol. 107, pp. 458–468, 2020.
- [103] Y. Pei, J. Zhang, G. Fletcher, and M. Pechenizkiy, "DyNMF: Role analytics in dynamic social networks," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3818–3824.
- [104] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, "Geom-GCN: Geometric graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–12.
- [105] S. Zhang, X. Liang, and J. Qi, "A review on role identification methods in social networks," *Chin. J. Comput.*, vol. 40, no. 3, pp. 649–673, 2017.
- [106] A. Zygumt, J. Koźlak, B. Gliwa, M. Stojkow, D. Żuchowska-Skiba, and Y. Demazeau, "Achieving and maintaining important roles in social media," *Inf. Process. Manage.*, vol. 57, no. 3, 2020, Art. no. 102223.
- [107] Y. Pei, F. Lyu, W. V. Ipenburg, and M. Pechenizkiy, "Subgraph anomaly detection in financial transaction networks," in *Proc. ACM Int. Conf. AI Finance*, 2020, pp. 1–18.

- [108] R. Rossi, B. Gallagher, J. Neville, and K. Henderson, "Role-dynamics: Fast mining of large dynamic networks," in *Proc. 21st Int. Conf. World Wide Web*, 2012, pp. 997–1006.
- [109] W. Peng, T. Varanka, A. Mostafa, H. Shi, and G. Zhao, "Hyperbolic deep neural networks: A survey," 2021, *arXiv:2101.04562*.
- [110] L. Wang, Y. Lu, C. Huang, and S. Vosoughi, "Embedding node structural role identity into hyperbolic space," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manag.*, 2020, pp. 2253–2256.



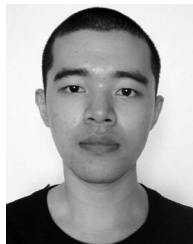
Pengfei Jiao received the PhD degree in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2021, he was a lecturer with the Center of Biosafety Research and Strategy, Tianjin University. He is currently a professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His research interests include complex network analysis and its applications.



Xuan Guo is currently working toward the doctoral degree with the College of Intelligence and Computing, Tianjin University, China. His research interests include complex network analysis, role discovery, network representation learning, and percolation model.



Ting Pan received the bachelor's degree from Xiamen University in 2020. She is currently working toward the master's degree with the School of Computer Science and Technology, Tianjin University. Her research interests include complex network analysis and role-based network representation learning.



Wang Zhang received the bachelor's degree in 2018 from Tianjin University, where he is currently working toward the master's degree with the School of Computer Science and Technology. His research interests include complex network analysis and network embedding.



Yulong Pei received the PhD degree in computer science from the Eindhoven University of Technology, Eindhoven, the Netherlands, in 2020. He is currently a postdoctoral researcher with the Department of Mathematics and Computer Science, Eindhoven University of Technology. His research interests include graph mining, social network analysis, and anomaly detection.



Lin Pan received the PhD degree in computer science from Tianjin University, Tianjin, China, in 2016. He is currently a lecturer with the School of Marine Science and Technology, Tianjin University, Tianjin, China. His research interests include complex network analysis, social computing, and marine data mining.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**